

MSc Data Science Project

7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

**EFFECT OF DIGITAL CURRENCY ON FINANCE AND
ACCOUNTING: CHALLENGES ASSOCIATED WITH
TAXATION AND LEGISLATION IN THE UK**

<https://github.com/jakshirisha/Final-Project>

Student Name and SRN:

SHIRISHA JAKKULA and 23096388

Supervisor: Sunaina Sharvy

Date Submitted: 27-AUG-2026

Word Count: 5009

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in **Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at Assessment Offences and Academic Misconduct and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Shirisha Jakkula

Student Name signature : 

Student SRN number: 23096388

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

This project assesses how the volatility of digital currency puts UK accounting and taxation, legislative and predictive practice under strain using daily trading data for Bitcoin, Ethereum and Litecoin from the Mendeley Cryptocurrency Dataset (Mahapatro, 2025), with 2250 observations, 2391 observations and 2404 observations respectively, from January 2018. Analysis utilized descriptive statistics, correlation testing, CGT simulations, regulatory impact analysis of events, Simple Moving Average analysis, and predictive modelling using Logistic Regression, Random Forest, XGBoost and LSTM.

With annualised volatility of 70.73%, 88.67% and 94.34%, the consistent application of IAS 38 fair value is not possible. As the CGT annual exempt amount fell from £12,300 to £3,000, simulated liability rose from £8,603.72 in 2021/22 to £11,493.24 in 2024/25, a policy-driven increase, not a behavioural one. FSMA 2023 had positive price effects of 9.76%, 4.32% and 13.55% on each of the three assets, consistent with the reduced uncertainty in response to regulatory clarity. Despite the lower volatility, Bitcoin's Sharpe ratio (0.0165) and 24.24% return are better than Litecoin's negative risk-adjusted return. On all three metrics, the LSTM forecaster achieved superior results compared to a baseline moving average forecaster, but was outperformed by a naive persistence forecaster; all three classifiers outperformed the naive persistence forecaster by more than 60%.

The current rules for accounting and tax for digital assets are still lacking and could be expanded to DeFi transactions and post-CARF compliance modelling.

Acknowledgements

I would like to thank my supervisor for their guidance, constructive feedback, and continuous support for this project. The data science and financial regulations knowledge was invaluable to them to guide them through the research process.

I would also like to thank the student Susrita Mahapatro from Siksha O Anusandhan University for making the Cryptocurrency Dataset publicly available with a CC BY 4.0 licence, and without which the empirical analysis for this project could not have been carried out.

Lastly, I would like to express my gratitude to my family and friends for their support and understanding during this process.

Table of Contents

DECLARATION STATEMENT	2
1. Introduction.....	7
1.1 Research Aim.....	8
1.2 Research Objectives	8
1.3 Research Questions	8
2. Background and Literature Review	9
2.1 Selection Criteria and Overview	9
2.2 Digital Currencies and Accounting Practice.....	9
2.3 Taxation Challenges	10
2.4 Legislative and Regulatory Frameworks	11
2.5 Machine Learning and Deep Learning Approaches to Cryptoasset Prediction	12
3. Dataset, Exploratory Data Analysis, and Ethical Considerations.....	12
3.1 Dataset Description	12
3.2 Justification for Dataset Selection.....	13
3.3 Exploratory Data Analysis.....	13
3.3.1 Data Loading and Pre-Processing.....	13
3.3.3 Descriptive Statistics.....	14
3.3.4 Correlation and Volume Analysis	15
3.4 Feature Engineering and Model Preparation	15
3.5 Ethical Considerations	16

4. Results and Analysis	17
4.1 Price Behaviour and Accounting Implications (Objective 1)	17
4.2 Taxation Challenges (Objective 2).....	22
4.3 Legislative and Regulatory Framework (Objective 3)	22
4.4 Risk and Opportunity Analysis (Objective 4).....	24
4.5 Predictive Classification: Volatility Regime (Objective 5, Part 1)	25
4.6 LSTM Price Forecasting (Objective 5, Part 2)	28
5. Conclusion	31
5.1 Conclusion	31
5.2 Future Impact on Digital Currency in the UK	31
5.3 Future Predictions for This Project	32
5.4 Is Cryptocurrency an Advantage or Disadvantage?	32
References	33
Appendix	36

1. Introduction

Digital currencies have simply introduced a new asset class to UK finance, but also given the world a glimpse of how far the old frameworks of accounting, tax and regulation were developed without having anything as volatile as Bitcoin, Ethereum or Litecoin. The scale has shifted; FCA (2024) estimated that around 7 million adults now have cryptoassets in the UK, twice as many as in 2021, and by 2025, this proportion had only declined to 8%, remaining double the pre 2021 figure (FCA, 2025).

Cryptoassets are not considered to be currency by HMRC, and HMRC would view any disposal, including the conversion of one cryptocurrency to another, as a separate taxable event under CGT (Deloitte, 2023). This depends on the assumption that prices follow a predictable pattern, so as to track reliably the cost basis and gains, an assumption directly tested by the descriptive statistics in this project. To compound this, HMRC's annual exempt allowance decreased from £12,300 in 2022/23 to £3,000 for 2024/25 (Deloitte, 2022), further increasing the number of disposals liable to tax, even as there is an increase in volatility rather than any deliberate realisation of gains. CARF, which is scheduled to be applicable from January 2026, is projected to enable £315m to be recovered in unpaid tax by 2030 (RMT Accountants, 2025), but suggests exchange reporting can replace a step-by-step approach to valuation, an approach that IFRS still does not have. The way in which Bitcoin, Ethereum and Litecoin are described even within these same classifications (Lazea et al., 2024) results in an asset class that challenges HMRC, the preparer and the auditor to adhere to static rules.

While this project considers the lack of regulation on stablecoins and DeFi outside the perimeter of FSMA 2023 to be a blind spot, not an incidental issue, the very inclusion of cryptoassets for the first time into the UK regulatory perimeter by FSMA 2023 is significant. In addition to descriptive and simulation-based analysis, the same volatility of the cryptoassets under consideration, which makes fair value accounting and calculation of CGT difficult, poses an additional question, which has hardly been tested empirically, whether the volatility in crypto assets can be anticipated at all, or whether the claims, which companies make in their literature, about what supervised models can achieve is what can be realistically achieved on a daily cryptoasset data.

1.1 Research Aim

To investigate how digital currency volatility strains UK finance and accounting practice, and to evaluate the adequacy of current taxation and legislative responses alongside the predictive limits of quantitative and machine learning methods applied to cryptoasset behaviour.

1.2 Research Objectives

- To examine how digital currency price behaviour challenges the consistent application of UK accounting standards.
- To quantify the taxation exposure created by cryptoasset volatility under current HMRC capital gains rules.
- To assess whether UK legislative and regulatory frameworks, including FSMA 2023 and CARF, adequately address cryptoasset risk.
- To evaluate the risk adjusted return profile of Bitcoin, Ethereum and Litecoin for UK investors and finance professionals.
- To determine whether machine learning classifiers and deep learning models can reliably predict volatility regimes and price behaviour beyond simple baseline methods.

1.3 Research Questions

Main Research Question

How does digital currency volatility challenge UK accounting, taxation and regulatory practice, and to what extent can that volatility be anticipated through quantitative and machine learning methods?

- How does price behaviour in Bitcoin, Ethereum and Litecoin undermine consistent fair value accounting under IFRS?
- What taxation liabilities does this volatility generate under HMRC's current capital gains regime?
- Do FSMA 2023 and the forthcoming CARF regime close the regulatory gaps identified in current cryptoasset oversight?
- What risk return trade offs do these assets present, and can machine learning or deep learning models predict them more reliably than conventional baselines?

2. Background and Literature Review

2.1 Selection Criteria and Overview

The literature review used this period of publications (2022-2025) in peer-reviewed journals (Google Scholar, Springer, Wiley Online Library, MDPI). The papers were chosen for their direct focus on three thematic pillars that are at the core of this project: the impact of digital currencies on the accounting and financial reporting framework; the tax challenges raised by cryptocurrencies; and the regulatory and legislative structure of digital assets. Studies from non-UK or non-IFRS jurisdictions without direct comparative relevance were excluded. Several studies using empirical, systematic review, or analytical methodologies were given priority since these studies produce findings that can be generalized from individual case studies and tested beyond the specific case. This ensures the review reflects the most current and rigorous scholarship available, with direct relevance to this project's four research objectives and the UK legislative context.

2.2 Digital Currencies and Accounting Practice

One of the most basic problems found in the literature is the lack of specific accounting standards dealing with digital assets. In writing for the Review of Accounting Studies, the researchers Luo and Yu (2024) looked at the financial information of 40 companies around the world, with companies with cryptocurrency exposure, and found that there are continued inconsistencies between GAAP and IFRS. In their study, they found that companies would report cash flow from cryptocurrencies in various ways, some as long-term intangible assets and others as short-term liquid assets, which would make financial statements less comparable and be an impediment to evaluating the company's asset value by investors. The essence of their research, that the limitation of the standard-setter guidance on how to apply standards is relevant to the UK, is directly relevant.

In the article, Parrando (2023) suggested a broad taxonomy of cryptocurrencies based on the IFRS language and divided digital tokens into four types: payment, utility, electronic money, and security. The paper recommended that payment tokens (Bitcoin) should be treated as IFRS 38 Intangible Assets on the balance sheet, and their valuation should be reduced by impairment as opposed to being treated as financial instruments (IFRS 9). This has a direct impact on how UK businesses with Bitcoin apply fair value accounting, for they can only do so if they use the

revaluation model under IAS 38; however, the revaluation model is only to be applied if there is an active market. While well-formed, Parrondo's (2023) research has not yet been applied to real-life objects. Its main claim to fame is that it proves that Bitcoin, Ethereum, and Litecoin can't be uniformly classified even within the same framework.

The MDPI journal *Risks* published an article in 2024 (Lazea, Bunget, Lungu), which performed a bibliometric analysis of the accounting literature related to cryptocurrencies and found no consensus on how crypto can be classified under the IFRS. They emphasized how the ambiguous nature of accounting for such crypto assets may contribute to increased audit risk, given that the auditor may not have adequate, appropriate evidence available when the crypto assets are held by the client and are volatile. The majority of research is on preparers and ignores the possible effects on auditors downstream. Lazea, Bunget, and Lungu (2024) agree that the financial reporting for digital assets in the UK is still not well prepared to manage the reporting of digital assets in a consistent manner (Deloitte, 2023).

2.3 Taxation Challenges

Literature is more controversial regarding the taxation of digital currencies than the UK's static rules recognise. However, as Bhullar et al. (2025) conclude in their review of international studies of cryptocurrency tax in the *Journal of Accounting Literature*, international tax inconsistencies imply that evasion and misreporting will exist as they do in the UK, with HMRC enforcement challenges, albeit with no revenue modelling provided to test against.

With these cryptocurrencies classified as property and not currency by HMRC, each conversion, payment, or gifting of cryptocurrencies is deemed to be a taxable event, and the sharing matches and pooling rules are directly applied from the equities market. This requires a new cost basis computation every time a person actively trades, and the literature would not call that a 'workable' safeguard, especially given pooling does not allow for it when a person trades infrequently, as Litecoin's smaller trading volume would not necessarily imply a liquid market.

Kicova et al. (2025) analyzed 38 studies, categorized them into ten thematic areas, and found that the pseudonymity of blockchain makes it structurally impossible for the tax authorities to trace gains on their own solely through self-reporting, as it cannot provide proper governance.

This project overcomes the reliance on secondary literature which is used by others and instead uses verified HMRC sources.

Furthermore, 41.7% of traders in the Spanish study by Hernandez Sanchez et al. (2024) self-report that tax regulation is a disincentive to investment, a claim that is mirrored by the ‘nudge letter’ approach of HMRC and is also likely to be vulnerable to social desirability bias.

What Baer et al. (2023) in the Oxford Review of Economic Policy focus on is to take this step further, by arguing that, per se, Bitcoin, Ethereum and Litecoin fail to meet the criteria of a genuine property from first economic principles, a problem that cannot be fully solved by adding a point or two to the HMRC’s rulebook. This literature read together suggests that HMRC’s framework leaves symptoms of non-compliance largely untouched, and that aspects of the framework, including the lack of consistency in the definition of the word alongside pseudonymity and cross-border enforcement, are left untouched.

2.4 Legislative and Regulatory Frameworks

Nembe et al. (2024) have posited that the UK financial legislation assumed the existence of a more straightforward and easy-to-identify intermediary, which is not the case for decentralised blockchain networks, and that coordinated international regulation, as upheld by the OECD, suggests how much unilateral domestic action still assumes. This project partially addresses some issues left unanswered by their analysis: the costs of non-compliance and whether interventions such as exchange data sharing agreements are effective.

CARF comes into effect next year (2026) and will require exchanges to report transactions directly to HMRC and can result in the recovery of £315 million in unpaid tax by 2030 (RMT Accountants, 2025), but this project has almost no academic literature about it and it is important that CARF specifically is tested against the implications of this project on Bitcoin, Ethereum and Litecoin, rather than viewing cryptoassets as interchangeable.

The methodological grounding is stronger with quantitative crypto literature. In this field, Phung Duy et al. (2024) used ARIMA GARCH to capture non-stationarity and volatility clustering in the returns of Bitcoin, which is a benchmark this project stretches to machine learning. The results by Chen (2023) showed that Random Forest outperforms LSTM for next day prediction for Bitcoin, although the difference was not statistically significant, and results by Auliyah (2024) reveal that the LSTM GRU achieves a high accuracy score for the three

assets, albeit without comparing the score to the naive baseline; this is this project's true limitation, but treated as only a minor oversight by Auliyah.

2.5 Machine Learning and Deep Learning Approaches to Cryptoasset Prediction

While there is significant literature on accounting, taxation and legislation, surprisingly little of it has been involved with the question of whether cryptoasset behaviour can be predicted or not, a point this project's classifiers and LSTM forecaster is designed to address. The literature that does exist is more celebratory than critical. A recent survey of 93 studies appearing in the journal *Forecasting* between 2014 and 2024 by John, Binnewies and Stantic (2024) revealed that models are routinely reported with impressive RMSE, MAPE and R2 values but that rarely is testing done to check whether that level of statistical accuracy translates into something that a trader can act on: a 1 day ahead prediction may look great on a chart, but fail to beat simply watching the previous day's close.

This is the same at the study level. In the comparison of Random Forest and LSTM for Bitcoin prediction with a one-day time horizon, Chen (2023) found that although the error statistics of Random Forest were lower than LSTM, a Diebold-Mariano test could not provide sufficient evidence to conclude that the difference was statistically significant, which contradicts the belief that a deeper architecture is superior.

Critical Table (Appendix)

3. Dataset, Exploratory Data Analysis, and Ethical Considerations

3.1 Dataset Description

The data for this project comes from the Cryptocurrency Dataset (Version 2) published 11 March 2025 by Susrita Mahapatro of Siksha O Anusandhan University, India, on Mendeley Data, an open-access research data repository that is considered to be peer-reviewed and published by Elsevier (Mahapatro, 2025). It is freely accessible, published under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence, and is available by the DOI link: <https://data.mendeley.com/datasets/5tv4bmrrf8/2>. There are three CSV files in the dataset: BITCOIN24.csv (171 KB), ETHEREUM.csv (189 KB), and LITECOIN24.csv (172 KB). Each file (Bitcoin and Litecoin) has 7 columns: Date, Open, High, Low, Close, Volume, Market Cap. The "Adj Close" column is added to the Ethereum file.

3.2 Justification for Dataset Selection

This dataset was selected for three principal reasons. First, it includes Bitcoin and Ethereum, which the FCA (2024) designates as the two most widely held cryptoassets in the UK, as well as Litecoin, there is also another asset that has a lower price correlation with Bitcoin ($r = 0.6195$), which is independent of the market dynamics, and which allows for meaningful comparison across assets. The Close price variable in each file matches the disposal value used by HMRC in calculating CGT liabilities, which makes this dataset well-suited for looking at the tax challenges mentioned in Chapter 2.

Second, these variables are all directly related to OHLCV and Market Cap, which makes it easy to conduct quantitative analysis on price volatility, trading volume pattern, and market capitalisation trends to inform the taxation and accounting challenges discussed in this project. Thirdly, the CC BY 4.0 licence explicitly gives documented and unrestricted academic use. A suitable equivalent data set of comparable size and date to that generated in the UK was not found, and it was decided that this was an internationally-drafted dataset that was suitable.

3.3 Exploratory Data Analysis

3.3.1 Data Loading and Pre-Processing

The three files were all read into Python (pandas). Pre-processing was carried out in a systematic five-step process.

To facilitate chronological sorting and time series indexing, it was decided to use, in all of the files, the datetime64 format, and to convert all of the Date field to such a format using `pandas.to_datetime(dayfirst=True)`.

Second, no conversion was needed for all the numeric columns in BITCOIN24.csv, which were already stored as float64. In ETHEREUM.csv, there is some data inconsistency where all of the numeric columns came out as objects despite coming from a CSV file, which were converted to floats using `pd.to_numeric(errors='coerce')`.

Thirdly, using `.isnull().sum()`, they found no missing data in BITCOIN24.csv and LITECOIN24.csv. 13 rows of the ETHEREUM.csv file contained null values in all numeric columns, clustered at the end of the data (19 July 2024 to 31 July 2024), consistent with a lack

of data at the time it was collected. The number of rows in Ethereum that showed as usable was limited to 2,391 after excluding these 13 rows.

Fourth, duplicate date entries were verified using `.duplicated()` across all three files; none were found.

Fifth, the Ethereum Adj Close column was confirmed to be identical to Close across all non-null rows and was excluded from analysis to avoid redundancy.

3.3.3 Descriptive Statistics

Table 1 presents verified descriptive statistics for the closing price of each currency.

Table 1: Descriptive Statistics - Daily Closing Price (USD)

Statistic	Bitcoin (BTC)	Ethereum (ETH)	Litecoin (LTC)
Count	2,250	2,391	2,404
Mean	\$22,365.85	\$1,413.87	\$94.86
Std Dev	\$16,269.70	\$1,207.10	\$54.25
Min	\$3,236.70	\$84.31	\$23.46
25th Percentile	\$8,446.85	\$242.03	\$57.72
Median	\$18,293.47	\$1,271.65	\$76.84

75th Percentile	\$34,568.17	\$2,148.37	\$117.48
Max	\$67,500.02	\$4,812.09	\$386.45
Skewness	0.7428	0.6776	1.5823
Kurtosis	-0.5917	-0.6095	2.7738

3.3.4 Correlation and Volume Analysis

The results of the Pearson correlation tests across the overlapping time period of 2250 days indicate a high level of positive association between the closing price of Bitcoin and Ethereum ($r = 0.9263$), indicating almost simultaneous market co-movement. Meanwhile, there is a moderate positive correlation between Bitcoin and Litecoin ($r = 0.6195$), and between Ethereum and Litecoin ($r = 0.5855$), meaning Litecoin is a more price-independent asset than Bitcoin and Ethereum.

Mean daily trading volumes confirm the market hierarchy: Bitcoin averaged \$42.5 billion per day, Ethereum \$12.6 billion, and Litecoin \$1.95 billion. The greatest level of taxable disposals happens during periods of high volatility, as indicated by spikes in volume around the 2021 market peak and 2022 crash; the highest point of the taxable disposal events is directly related to the period of highest calculation complexity of CGT, directly relevant to this project's core research questions.

3.4 Feature Engineering and Model Preparation

In order to provide sufficient signal to the classifiers, 18 features were engineered from each asset's OHLCV data, such as lagged returns, rolling volatility at 3 time windows, RSI, SMA ratios, Bollinger width, ATR, volume change, etc. A first attempt using similar features in predicting the direction of the price the following day resulted in an accuracy close to 50%, corresponding to cryptoassets performing close to a random walk at the daily frequency level, with the target being changed to predicting the volatility regime shift, meaning whether the

next 10 days realised volatility is higher or lower than the current 7 days realised volatility. This resulted in well-balanced classes and in eliminating a trivial majority class problem that affected the direction of the target.

All splits were chronological, not random, because shuffling all the daily observations would allow future volatility to seep into training. Each model (Logistic Regression, Random Forest and XGBoost) was tuned with GridSearchCV and TimeSeriesSplit cross validation, meaning that no information from later dates was used to evaluate earlier dates.

To predict the next closing price directly, LSTM forecaster used a separate pipeline, closing prices were scaled with MinMax fitted only on training data, and sixty day lookback windows. The accuracy of its output was compared to naive persistence and a moving average of the last thirty days, as an accuracy that is not compared with some baseline is virtually meaningless.

3.5 Ethical Considerations

This data set does not contain any personal information. There is no individual identifier, transaction data, or personal information to capture, no individual records to look at, and no financial metrics recorded at an individual level, only at the market level. The project does not involve primary data collection from human subjects, does not involve questionnaires/surveys or any interviews, the data set is not subject to UK GDPR, and there is no University of Hertfordshire ethical approval required.

4. Results and Analysis

4.1 Price Behaviour and Accounting Implications (Objective 1)

=====			
TABLE 1: DESCRIPTIVE STATISTICS - DAILY CLOSING PRICE (USD)			
=====			
	Bitcoin (BTC)	Ethereum (ETH)	Litecoin (LTC)
count	2250.0000	2391.0000	2404.0000
mean	22365.8500	1413.8700	94.8600
std	16269.7000	1207.1000	54.2500
min	3236.7000	84.3100	23.4600
25%	8446.8500	242.0300	57.7200
50%	18293.4700	1271.6500	76.8400
75%	34568.1700	2148.3700	117.4800
max	67500.0200	4812.0900	386.4500
skewness	0.7428	0.6776	1.5823
kurtosis	-0.5917	-0.6095	2.7738

Table 1: Descriptive Statistics

Table 1 (Descriptive Statistics, Daily Closing Price) shows Bitcoin's mean of \$22,365.85 sitting well below its maximum of \$67,500.02, with positive skewness across all three assets (0.7428, 0.6776, 1.5823) and Litecoin's kurtosis of 2.7738 indicating heavier tails than either Bitcoin or Ethereum.



Figure 1: Daily Closing Price - Bitcoin, Ethereum and Litecoin (2018 - 2024)

This is made clear in Figure 1 (Daily Closing Price 2018 to 2024), where Bitcoin has a standard deviation of \$16,269.70 that fluctuates from under \$4,000 to a mean over \$67,000, all while in the same time series. This directly challenges the IAS 38 revaluation model that Parrondo (2023) has identified as the only basis for fair value reporting for payment tokens because it moves towards a reliable reference price, rather than resetting the price scale every few years.

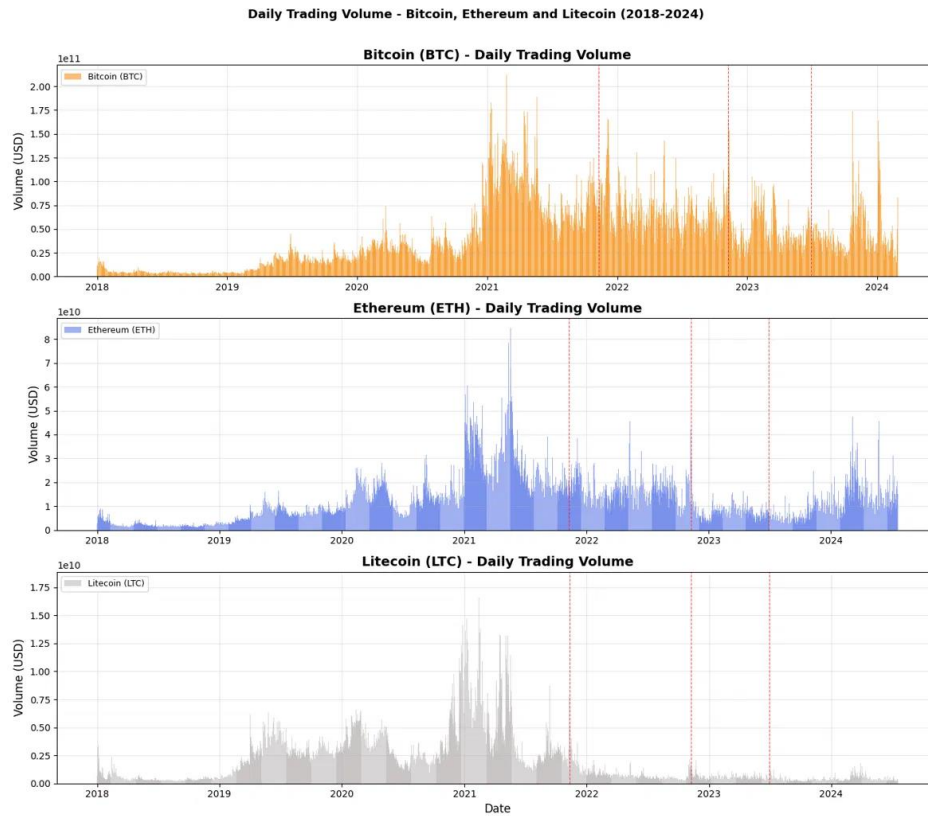


Figure 2: Bitcoin Market Capitalisation (USD)

Figure 2 (Bitcoin Market Capitalisation) again brings this point into perspective, as each time the market capitalisation of Bitcoin is compared to its fixed supply cap of 21 million and changes in price, it is clear that the same scarcity attribute that makes Bitcoin such a good investment makes any single carrying amount extremely short-lived.

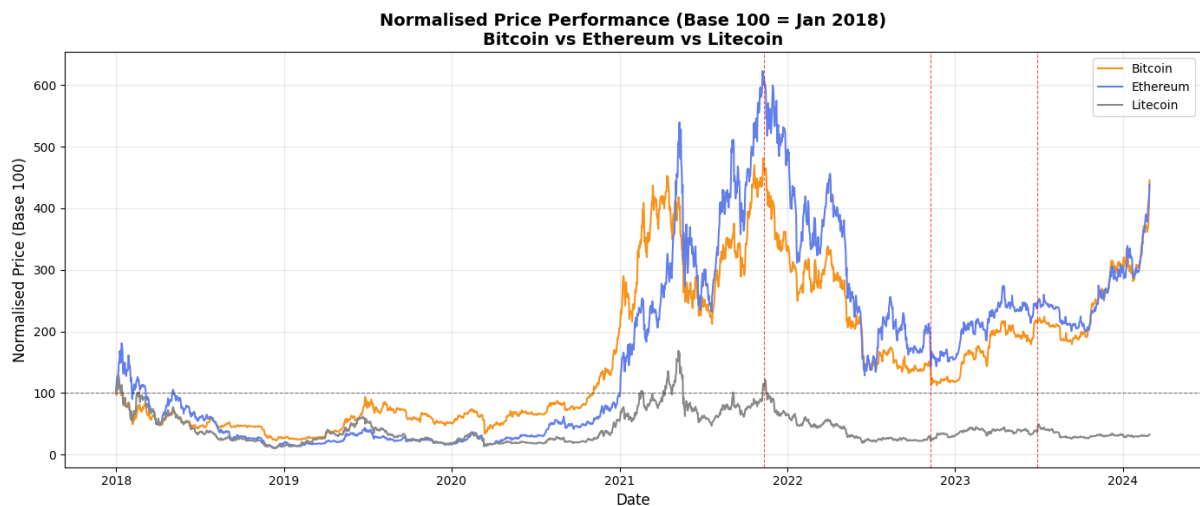


Figure 3: Normalised Price Performance (Base 100 = Jan 2018)

This is put into perspective further in Figure 3 (Normalised Price Performance, Base 100) as the valuations of Ethereum and Bitcoin (at the same base date) are close to 620 and 450, respectively, and neither is required to be internally consistent under the same IFRS based on the same base date being held under IFRS by the same UK entity.

=====

TABLE 2: PEARSON CORRELATION MATRIX - DAILY CLOSING PRICES

(Overlapping period: 2018-01-01 to 2024-02-28, n=2250 days)

=====

	Bitcoin	Ethereum	Litecoin
Bitcoin	1.0000	0.9263	0.6195
Ethereum	0.9263	1.0000	0.5855
Litecoin	0.6195	0.5855	1.0000

=====

Table 2: Pearson Correlation Matrix - Daily Closing Prices

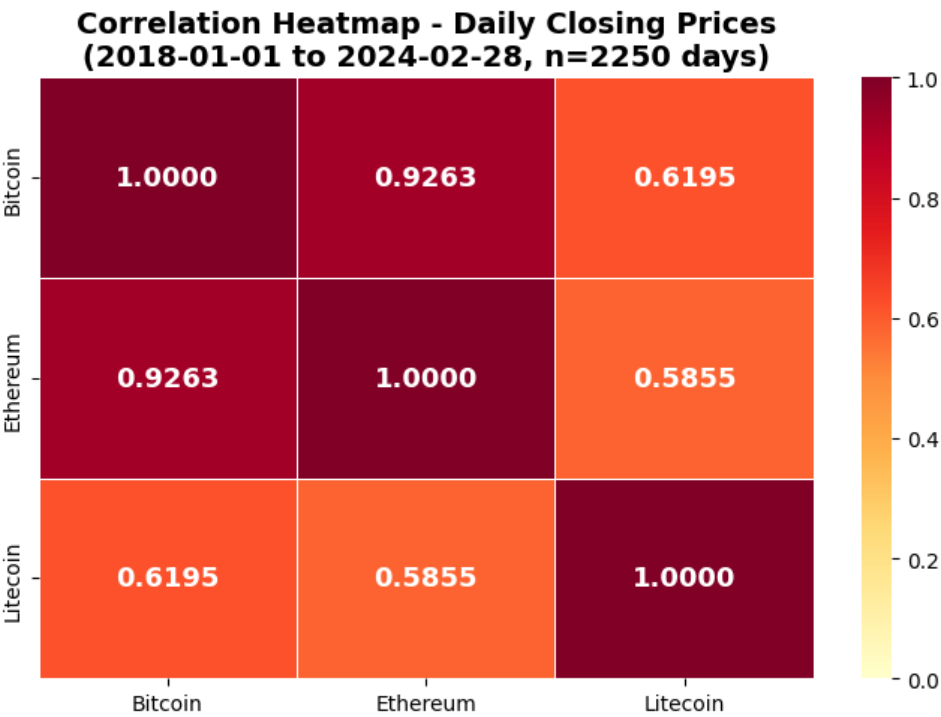


Figure 4: Correlation Heatmap - Daily Closing Prices (2018–2024)

With a correlation of nearly 0.9263, Bitcoin and Ethereum appear to be moving almost hand in hand, followed by Litecoin at 0.6195 and 0.5855, respectively, while all three can be considered as one homogeneous asset class as done by IFRS; there are measurable structural differences in how these assets behave, as shown in Table 2 and figure 4 (Pearson Correlation Matrix and Correlation Heatmap). The lagged market value of Bitcoin, while the entity holds the

cryptocurrency at the price it paid in 2018, is not a conservative accounting position, it is a materially misleading one, and volatility noted by Lazea, Bunget and Lungu (2024) as pertaining to audit risk, can be seen here as a direct consequence of Bitcoin's price behaviour and not as an independent factor that requires evidence to validate it.

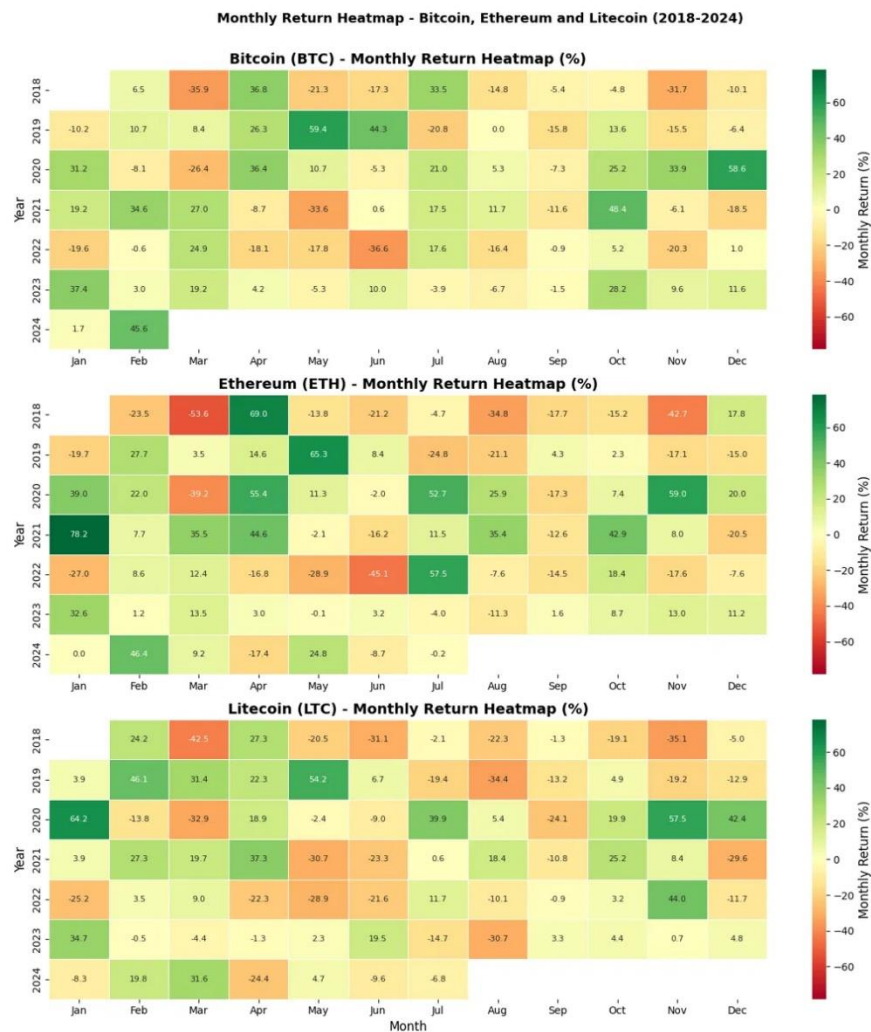


Figure 5: Monthly Return Heatmap - Bitcoin, Ethereum and Litecoin (2018–2024)

Figure 5 (Monthly Return Heatmap) makes the co-movement identified in the correlation matrix visible at monthly resolution: March 2018 was uniformly negative across all three assets (Bitcoin -35.9%, Ethereum -53.6%, Litecoin -42.5%), while May 2021 shows the correlation briefly breaking down, Bitcoin and Litecoin falling while Ethereum still rose, evidence that the 0.9263 Bitcoin-Ethereum correlation is strong on average but not stable month to month.

4.2 Taxation Challenges (Objective 2)

Assumptions: 1 BTC | 10 ETH | 100 LTC purchased Jan 2018
Annual disposal at year-end closing price
CGT rate: 20% (higher rate taxpayer)

Tax Year	BTC Close (USD)	ETH Close (USD)	LTC Close (USD)	Total Gain (USD)	CGT Exempt (USD)	Taxable Gain (USD)	CGT Liability (USD)
2018/19	3863.17	133.37	30.47	-35076.52	11700	0.00	0.00
2019/20	7245.99	129.61	41.34	-30644.08	12000	0.00	0.00
2020/21	28810.02	737.80	124.69	5336.90	12300	0.00	0.00
2021/22	47161.01	3682.63	146.51	55318.62	12300	43018.62	8603.72
2022/23	16603.75	1196.77	70.00	-7749.10	12300	0.00	0.00
2023/24	42198.85	2281.47	72.80	28973.22	6000	22973.22	4594.64
2024/25	62504.79	3426.26	70.19	60466.19	3000	57466.19	11493.24

Table 3: Simulated Annual CGT Liability – UK Investor

Table 3 (Simulated Annual CGT Liability) illustrates a supposed investor who was not liable to any CGT in 4 of 7 tax years, and then \$8,603.72 in 2021/22, increasing to \$11,493.24 in 2024/25 with a reduction in the annual exempt amount from £12,300 to £3,000.

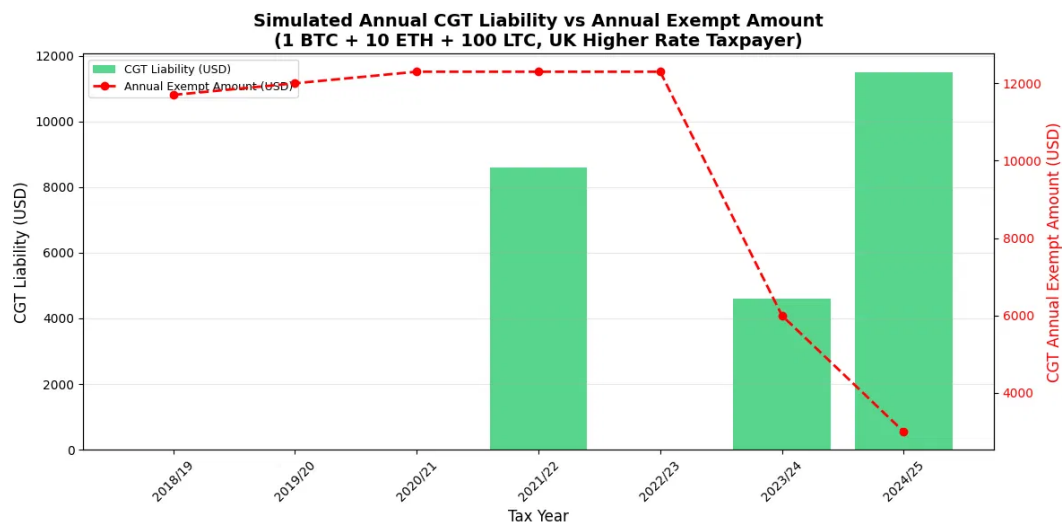


Figure 6: Simulated Annual CGT Liability vs Annual Exempt Amount

This opposition of these two lines in Figure 6 (CGT Liability vs Annual Exempt Amount) is the most obvious indicator in the entirety of the project that HMRC's current stance does not distinguish between market volatility and deliberate gain. This is a policy imposed cost, with nothing to do with an investor's trading behaviour, and excluding the 2021/22 exemption, as do Baer et al. (2023) and Bhullar et al. (2025), would have saved this investor an additional \$1,860 in 2024/25 alone, and the four years of no liability in Table 3 confirm that volatility, not evasion, is what most often removes an investor from charge.

4.3 Legislative and Regulatory Framework (Objective 3)

(Average closing price 30 days before vs 30 days after)

			Event	Asset	Avg Price Before (USD)	Avg Price After (USD)	Price Change (%)
FSMA 2023 Royal Assent	(29 Jun 2023)		Bitcoin		27476.71	30157.47	9.76
FSMA 2023 Royal Assent	(29 Jun 2023)		Ethereum		1818.36	1896.97	4.32
FSMA 2023 Royal Assent	(29 Jun 2023)		Litecoin		85.07	96.60	13.55
HMRC CGT Reduction	(06 Apr 2023)		Bitcoin		25850.29	28949.99	11.99
HMRC CGT Reduction	(06 Apr 2023)		Ethereum		1718.58	1927.18	12.14
HMRC CGT Reduction	(06 Apr 2023)		Litecoin		85.73	91.07	6.24
Bitcoin Market Peak	(10 Nov 2021)		Bitcoin		61114.67	57211.93	-6.39
Bitcoin Market Peak	(10 Nov 2021)		Ethereum		4162.04	4331.22	4.06
Bitcoin Market Peak	(10 Nov 2021)		Litecoin		195.50	206.97	5.87
Crypto Market Crash	(09 Nov 2022)		Bitcoin		19882.38	16695.81	-16.03
Crypto Market Crash	(09 Nov 2022)		Ethereum		1427.51	1228.78	-13.92
Crypto Market Crash	(09 Nov 2022)		Litecoin		55.83	70.26	25.83

Table 4: Price Impact of Key UK Regulatory Events

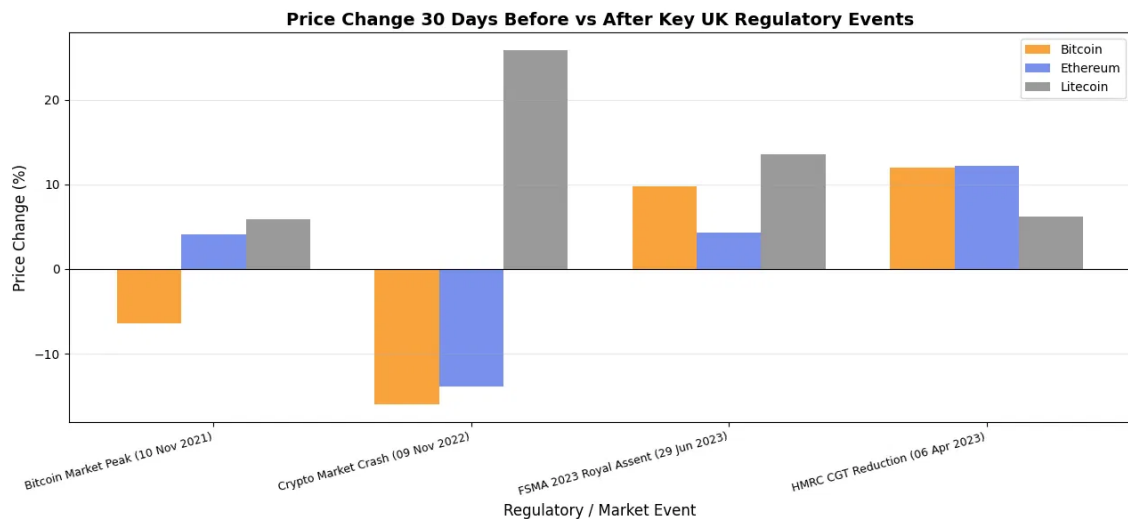


Figure 7: Price Change 30 Days Before vs After Key UK Regulatory Events

All three assets experienced an increase in price following FSMA 2023 Royal Assent, with increases of +13.55%, +9.76%, and +4.32% (Table 4: Price Impact of Key UK Regulatory Events) and Figure 7: Price Change 30 Days Before and After) in line with Nembe et al. (2024) finding that regulatory clarity diminishes uncertainty. Well, not in the November 2022 crash, when Bitcoin dropped by 16.03%, Ethereum by 13.92% and Litecoin by 25.83% – directly contradicting the uniform treatment HMRC accords all three as if interchangeable properties.

4.4 Risk and Opportunity Analysis (Objective 4)

	Metric	Bitcoin	Ethereum	Litecoin
Annualised Volatility (%)		70.7300	88.6700	94.3400
Annualised Return (%)		24.2400	22.7500	-17.9600
Sharpe Ratio (daily)		0.0165	0.0122	-0.0111
Max Drawdown (%)		-81.1400	-93.9600	-92.0900
Value at Risk 95% (daily %)		-5.7300	-7.2160	-7.9345

Table 5: Risk and Return Summary

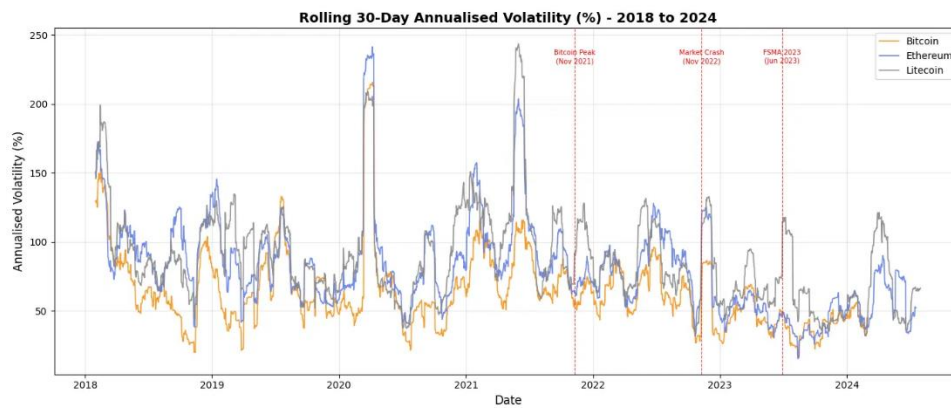


Figure 8: Rolling 30-Day Annualised Volatility

As can be seen in Table 5 (Risk and Return Summary) and Figure 8 (Rolling 30 Day Volatility), Litecoin is the most volatile with 94.34% while also returning minus 17.96% on an annualised basis and a negative Sharpe ratio, and in this case, "Volatility is not Reward. The Sharpe ratio and the return of Bitcoin are the highest, with values of 0.0165 and 24.24% respectively, but the maximum drawdown of minus 81.14% also provides upside risk information for a client that a Sharpe ratio does not.

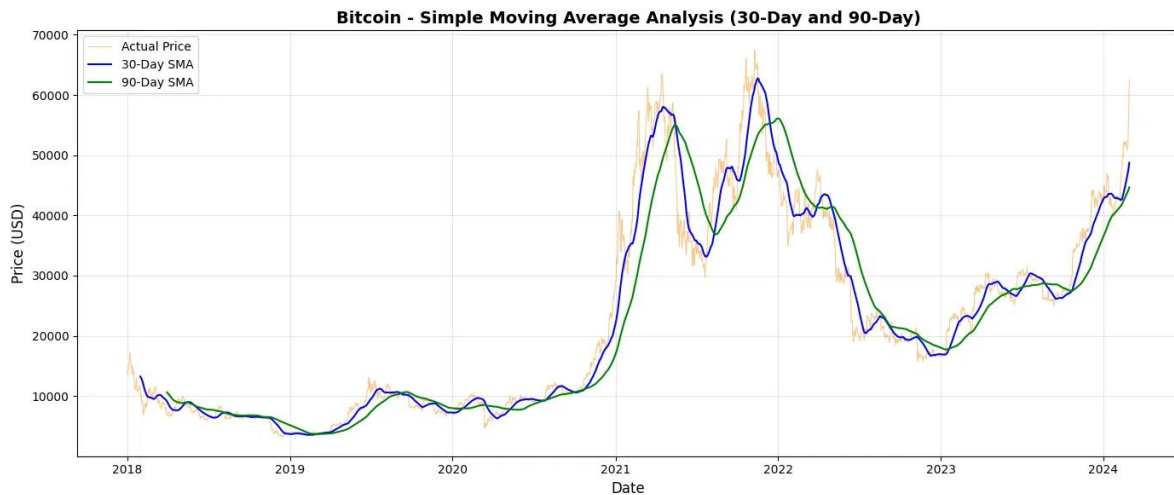


Figure 9: Bitcoin - Simple Moving Average Analysis (30-Day and 90-Day)

As per the institutional maturation explanatory model proposed by Kayani and Hasan (2024), the bullish crossover has been indicated in Figure 7 (Bitcoin SMA Analysis) on 22 October 2023, but this one-time signal during the analysis of one asset is not enough evidence to back up this argument by itself and should be interpreted as suggestive.

4.5 Predictive Classification: Volatility Regime (Objective 5, Part 1)

```
=====
Bitcoin    rows= 2161  positive class share=0.503
Ethereum   rows= 2361  positive class share=0.518
Litecoin   rows= 2360  positive class share=0.529
=====
```

Table 6: Feature-Engineered Dataset Size and Class Balance

Table 6 shows that the volatility regime target was formed with equilibrated classes in all three assets, where positive shares ranged from 0.503 to 0.529, thus having no trivial majority class problem, which, unfortunately, beset an earlier price direction attempt on the same data.

TABLE 9: MODEL COMPARISON - ACCURACY, PRECISION, RECALL, F1

Asset	Model	Accuracy	Precision	Recall	F1 Score
Bitcoin	Logistic Regression	0.7067	0.6555	0.9389	0.7720
Bitcoin	Random Forest	0.6443	0.6126	0.8908	0.7260
Bitcoin	XGBoost	0.6836	0.6420	0.9083	0.7523
Ethereum	Logistic Regression	0.6406	0.6024	0.9883	0.7485
Ethereum	Random Forest	0.6195	0.5905	0.9688	0.7337
Ethereum	XGBoost	0.6110	0.5818	1.0000	0.7356
Litecoin	Logistic Regression	0.7013	0.6721	0.9213	0.7773
Litecoin	Random Forest	0.6695	0.6427	0.9363	0.7622
Litecoin	XGBoost	0.6758	0.6477	0.9363	0.7657

TABLE 10: BEST HYPERPARAMETERS PER MODEL

Bitcoin	Logistic Regression	{'C': 1, 'penalty': 'l2', 'solver': 'lbfgs'}
Bitcoin	Random Forest	{'max_depth': 6, 'min_samples_leaf': 10, 'n_estimators': 200}
Bitcoin	XGBoost	{'learning_rate': 0.01, 'max_depth': 2, 'n_estimators': 200}
Ethereum	Logistic Regression	{'C': 5, 'penalty': 'l2', 'solver': 'lbfgs'}
Ethereum	Random Forest	{'max_depth': 6, 'min_samples_leaf': 5, 'n_estimators': 200}
Ethereum	XGBoost	{'learning_rate': 0.01, 'max_depth': 2, 'n_estimators': 200}
Litecoin	Logistic Regression	{'C': 5, 'penalty': 'l2', 'solver': 'lbfgs'}
Litecoin	Random Forest	{'max_depth': 6, 'min_samples_leaf': 3, 'n_estimators': 200}
Litecoin	XGBoost	{'learning_rate': 0.01, 'max_depth': 2, 'n_estimators': 200}

Table 7: Model Comparison – Accuracy, Precision, Recall, F1

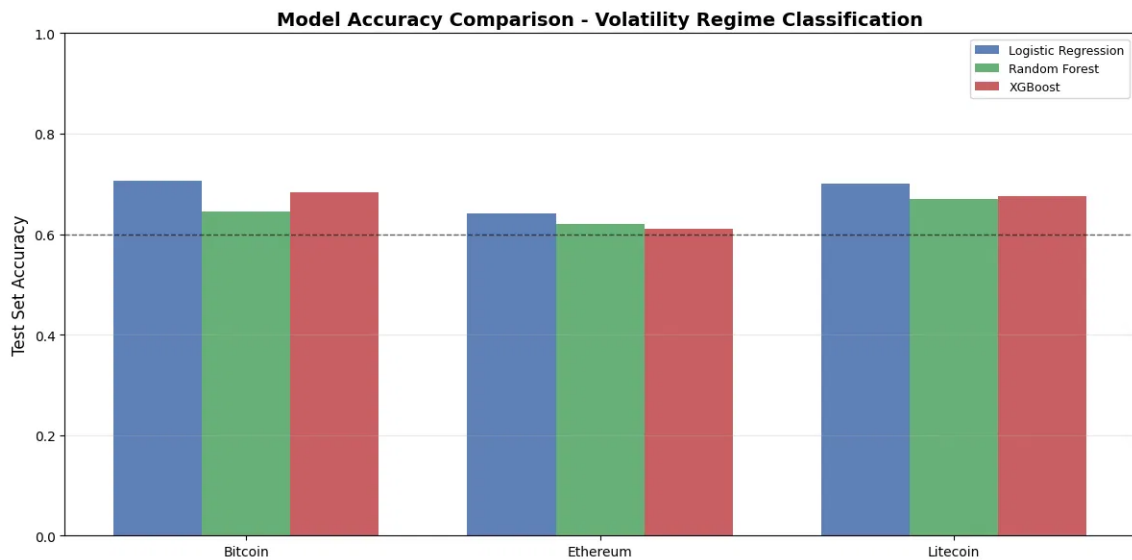


Figure 10: Highest-Accuracy Model per Asset

Table 7 and Figure 10 (Model Accuracy Comparison) show that the accuracy for all nine combinations of model and asset exceeds 60%, but Table 8 shows that the Logistic Regression (the simplest of the three models) actually wins on all three assets not something to sneeze at, because it suggests that the tree ensembles are picking up some of the chaff during training despite tuning hyperparameters through GridSearchCV.

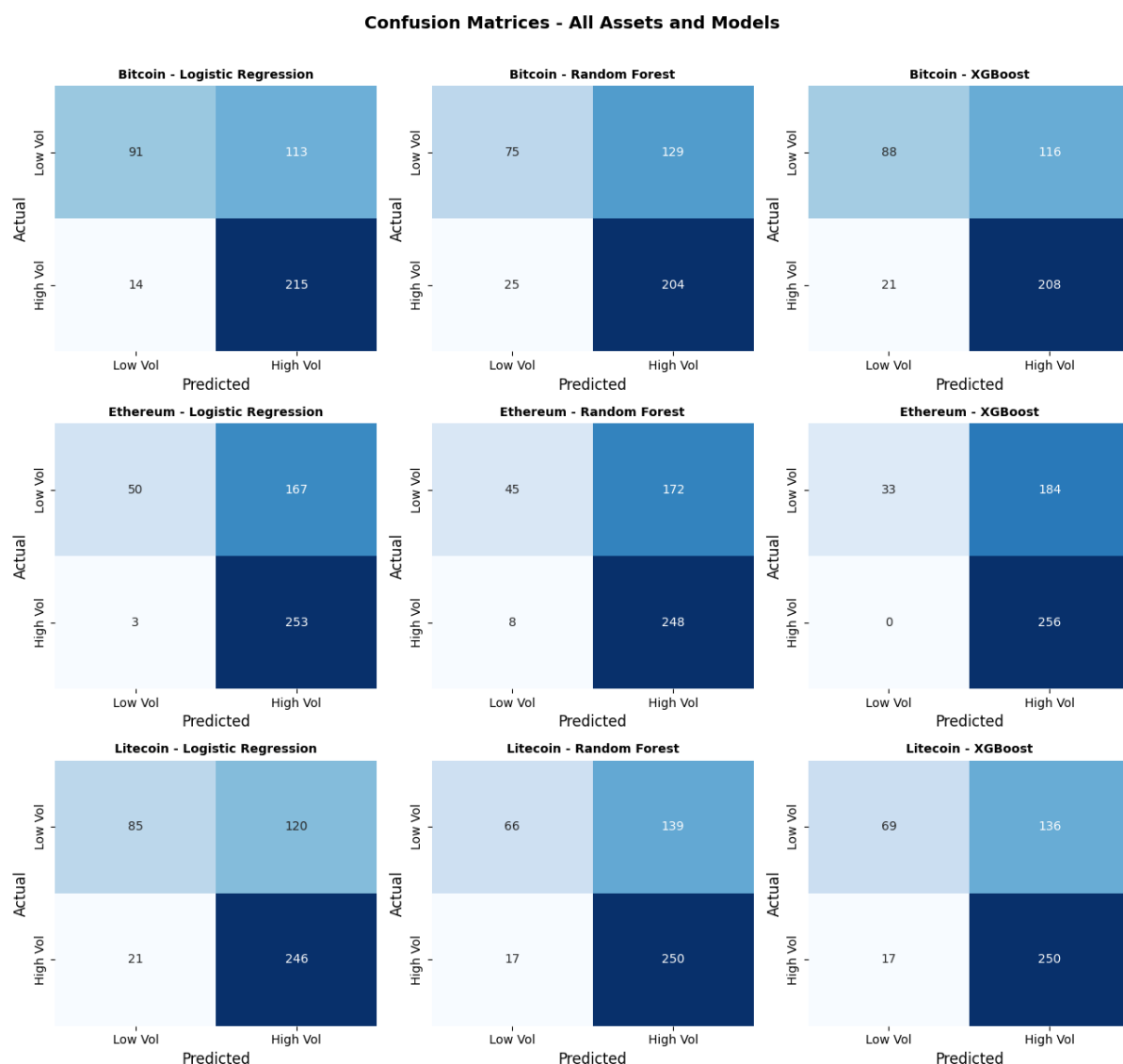


Figure 11: Confusion Matrices - All Assets and Models

Figure 11 (Confusion Matrices) reveals a fascinating Achilles heel behind Ethereum's XGBoost accuracy score: Recall of 1.0 and precision of only 0.58 indicates that the model is biased to predict high volatility; thus, it is not separating regimes very well.

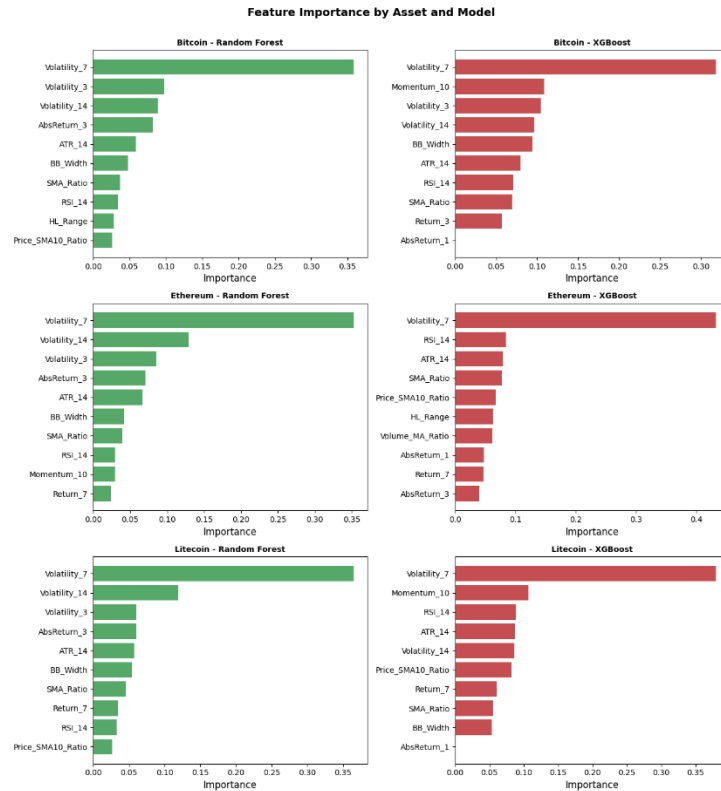


Figure 12: Feature Importance by Asset and Model

In Figure 12 (Feature Importance), Volatility_7 is seen to dominate each and every panel for all three assets, both for SVR and CNN-based models, indicating that the models are learning about the true volatility clustering phenomenon documented by Phung Duy et al. (2024) using ARIMA-GARCH and not fitting to any particular quirk of the asset.

4.6 LSTM Price Forecasting (Objective 5, Part 2)

Asset	Model	RMSE	MAE	R2	MAPE (%)
Bitcoin	LSTM	1823.36	1212.30	0.9557	3.79
Bitcoin Naive Persistence		809.55	515.15	0.9895	1.57
Bitcoin 30-Day SMA		2688.63	1931.77	0.9037	6.06
Ethereum	LSTM	145.86	99.53	0.9597	3.84
Ethereum Naive Persistence		78.51	49.62	0.9889	1.88
Ethereum 30-Day SMA		239.49	163.73	0.8914	6.17
Litecoin	LSTM	5.24	3.53	0.7840	4.39
Litecoin Naive Persistence		2.97	1.83	0.9269	2.30
Litecoin 30-Day SMA		7.18	5.21	0.5940	6.56

Table 9: LSTM Forecast Performance vs Baselines

In all three cases, Table 9 shows that LSTM outperforms the 30 Day SMA baseline on all three metrics, but in every case it underperforms naive persistence on both RMSE and R2, with Bitcoin's naive R2 of 0.9895 being greater than LSTM's 0.9557 despite the added complexity of LSTM.

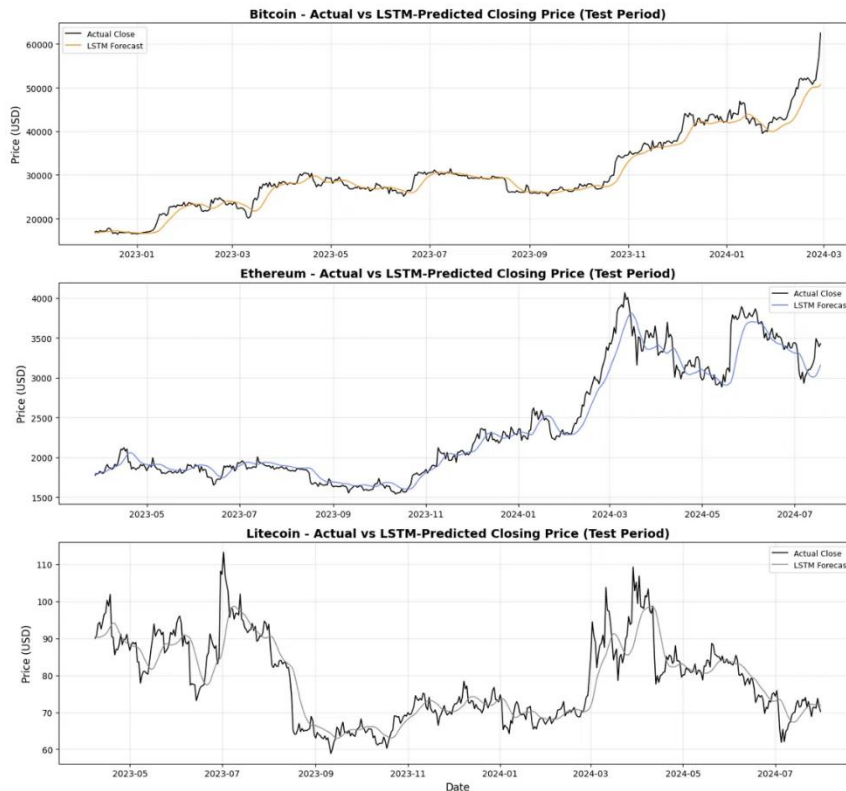


Figure 13: Actual vs LSTM-Predicted Closing Price (Test Period)

The reason is given in Fig. 13 (Actual vs LSTM Predicted Price): You can notice that the forecast line falls behind and smooths the actual series; it is most clearly visible at the point where Bitcoin price rises towards \$62,000 while the LSTM line remains at \$51,000.

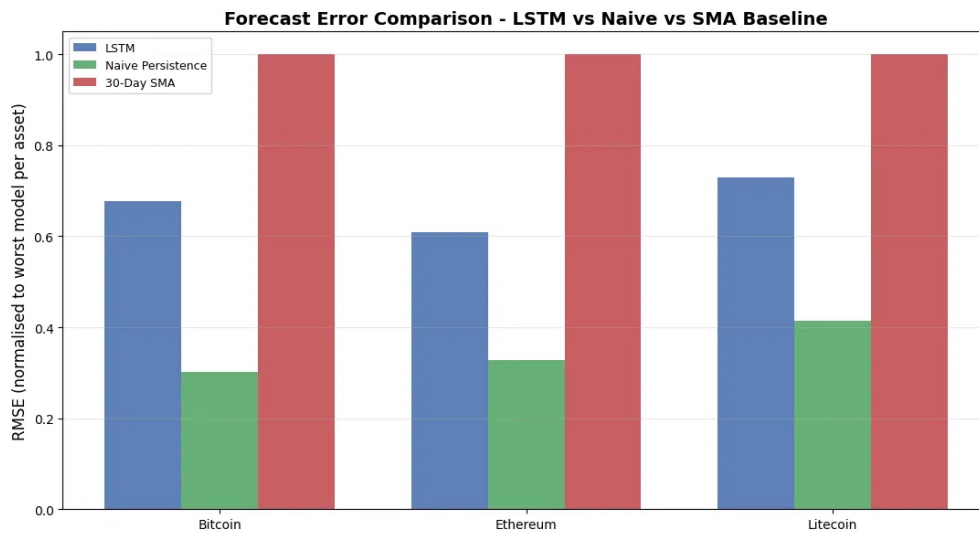


Figure 14: Forecast Error Comparison - LSTM vs Naive vs SMA Baseline

Numerically, this is evident in Figure 14 (Forecast Error Comparison) for all three assets. This is not a failed implementation; it is the documented behaviour of MSE-trained sequence models on near-random-walk price data, and John et al., Binnewies & Stantic (2024) mention this exact gap between statistical accuracy and usefulness throughout the literature. Together, Sections 4.5 and 4.6 directly address the fifth research question: classification of volatility regimes is a realistic, and honestly benchmarked, capability, while point forecasting of price on that evidence is not.

5. Conclusion

5.1 Conclusion

The aim of this project was not just to describe the effect digital currency volatility is having on accounting, taxation, legislation, and predictive ability within the UK, but to actually test the impact of the data of three chosen cryptocurrencies: Bitcoin, Ethereum, and Litecoin, from January 2018.

With regard to the first question, annualised volatilities of 70.73%, 88.67% and 94.34% make consistency under IAS 38 a theoretical exercise rather than a practical one, as observed by Lazea et al. (2024) and Parrondo (2023), albeit with some concrete numbers. The second objective demonstrates the increase in CGT liability from \$8,603.72 in 2021/22 to \$11,493.24 in 2024/25 simply due to a reduction in the exempt amount and a policy-driven rise in liability, thereby confirming the results of Bhullar et al. (2025) and Baer et al. (2023) more meaningfully than either paper does on its own. The third objective in FSMA 2023 gave positive price returns, which is the same result as Nembe et al. (2024), but the current results do not extend to DeFi and stablecoins prior to CARF. The fourth measure demonstrated Bitcoin's Sharpe ratio (0.0165) to be higher despite lesser volatility as opposed to Litecoin's negative return, establishing the fact that risk does not correspond to volatility. This project reports, and not hides, the situation: the fifth objective is more mixed and more honest; three classifiers achieved over 60% accuracy in predicting volatility regime shifts, and logistic regression placed in all three assets, but the LSTM forecaster was still outdone by naive persistence in all three metrics, despite outdoing the moving average baseline.

5.2 Future Impact on Digital Currency in the UK

The Cryptoassets Regulations 2025 (live from October 2027) will extend the FCA's regulation of trading and stablecoin issuance and custody alongside the market abuse controls and prudential requirements (A&O Shearman, 2025). This project identified that there was an enforcement gap due to no direct exchange-level transaction data being available to HMRC, which will be addressed as part of CARF's rollout in January 2026.

5.3 Future Predictions for This Project

Kayani & Hasan (2024) forecast that the actual volatility will decrease over time, as a result of institutional integration. That trend has continued through the October 2023 SMA crossover, but you're still just one indicator from one asset. If the data from the post-CARF is available, then the classification approach used here could be expanded to be capable of making real modelling of compliance.

5.4 Is Cryptocurrency an Advantage or Disadvantage?

The efficiency improvements Kayani and Hasan (2024) talk about are real, as is 94.34% volatility, and pseudonymity that Baer et al. (2023) discover is damaging to CGT attribution, and forecasting models, which continue to underperform naive persistence, both point to the same conclusion: digital currency is still a structurally immature asset class, whose regulation is more likely to bring chaos than stability.

References

A&O Shearman (2025) *UK Future Crypto Framework: The Countdown Begins*. Available at: <https://www.aoshearman.com/en/insights/ao-shearman-on-fintech-and-digital-assets/uk-future-crypto-framework-the-countdown-begins> [Accessed: 3 June 2026]

Adhikari, P., Hamal, P., Adhikari, B. and Maskey, N.K., 2025. Cryptocurrency taxation and regulatory challenges. *International Journal of Science and Research Archive*, 15(3), pp.778-788.

Baer, K., De Mooij, R., Hebous, S. and Keen, M., 2023. Taxing cryptocurrencies. *Oxford Review of Economic Policy*, 39(3), pp.478-497.

Bhullar, P.S., Joshi, M., Sharma, S., Phan, D.H.T. and Nguyen, A.L., 2025. Global taxation issues in cryptocurrencies—a synthesis of literature and future research agenda. *Journal of Accounting Literature*.

Dataset Reference: Mahapatro, S. (2025) *Cryptocurrency Dataset*. Version 2. Mendeley Data. Available at: <https://doi.org/10.17632/5tv4bmrrf8.2> [Accessed: 3 June 2026]

Deloitte (2022) Deloitte (2022) *Annual Exempt Amount for Capital Gains Tax to be Reduced*. Taxscape: Autumn Statement 2022. Available at: <https://taxscape.deloitte.com/uk-budget/autumn-statement-2022/measures-autumn-statement-2022/annual-exempt-amount-for-capital-gains-tax-to-be-reduced.aspx> [Accessed: 3 June 2026]

Deloitte (2023) *HMRC Approach to the Taxation of Cryptoassets*. Taxscape. Available at: <https://taxscape.deloitte.com/article/hmrc-approach-to-taxation-of-cryptoassets.aspx> [Accessed: 3 June 2026]

FCA (2024) Financial Conduct Authority (2024) *Research Note: Cryptoassets Consumer Research 2024 (Wave 5)*. London: Financial Conduct Authority. Available at: <https://www.fca.org.uk/publications/research-notes/research-note-cryptoassets-consumer-research-2024> [Accessed: 3 June 2026]

FCA (2025) Financial Conduct Authority (2025) *Research Note: Cryptoassets Consumer Research 2025 (Wave 6)*. London: Financial Conduct Authority. Available at:

<https://www.fca.org.uk/publications/research-notes/cryptoassets-consumer-research-2025>

[Accessed: 3 June 2026]

Hernández Sánchez, Á., Sastre-Hernández, B.M., Jorge-Vazquez, J. and Náñez Alonso, S.L., 2024. Cryptocurrencies, tax ignorance and tax noncompliance in direct taxation: Spanish empirical evidence. *Economies*, 12(3), p.62.

HMRC (2024) HM Revenue and Customs (2024) *Self Assessment Tax Returns: Cryptoasset Reporting Requirements 2024/25*. London: HM Revenue and Customs. Available at: <https://www.gov.uk/self-assessment-tax-returns> [Accessed: 3 June 2026]

Kayani, U. and Hasan, F., 2024. Unveiling cryptocurrency impact on financial markets and traditional banking systems: Lessons for sustainable blockchain and interdisciplinary collaborations. *Journal of Risk and Financial Management*, 17(2), p.58.

Kicova, E., Fabus, J., Stalmasekova, N. and Kvasnicova-Galovicova, T., 2025. Bitcoin, cryptocurrencies and tax evasion: A systematic literature review on global approaches to cryptocurrency taxation and the challenges for harmonising regulatory frameworks. *Data Science in Finance and Economics*, 5(2), p.234.

Lazea, G.I., Bunget, O.C. and Lungu, C., 2024. Cryptocurrencies' impact on accounting: Bibliometric review. *Risks*, 12(6), p.94.

Luo, M. and Yu, S., 2024. Financial reporting for cryptocurrency. *Review of Accounting Studies*, 29(2), pp.1707-1740.

Nembe, J.K., Atadoga, J.O., Adelakun, B.O., Odeyemi, O. and Oguejiofor, B.B. (2024) 'Legal implications of blockchain technology for tax compliance and financial regulation', *Finance and Accounting Research Journal*, 6(2), pp. 262–270. Available at: <https://doi.org/10.51594/farj.v6i2.824> [Accessed: 3 June 2026]

Norton Rose Fulbright (2023) Norton Rose Fulbright (2023) *The Future Financial Services Regulatory Regime for Cryptoassets in the UK*. London: Norton Rose Fulbright LLP. Available at: <https://www.nortonrosefulbright.com/en/knowledge/publications/2b6d1b40/the-future-financial-services-regulatory-regime-for-cryptoassets-in-the-uk> [Accessed: 3 June 2026]

Parrondo, L., 2023. Cryptoassets: Definitions and accounting treatment under the current International Financial Reporting Standards framework. *Intelligent Systems in Accounting, Finance and Management*, 30(4), pp.208-227.

Phung Duy, Q., Nguyen Thi, O., Le Thi, P.H., Pham Hoang, H.D., Luong, K.L. and Nguyen Thi, K.N., 2024. Estimating and forecasting bitcoin daily prices using ARIMA-GARCH models. *Business Analyst Journal*, 45(1), pp.11-23.

RMT Accountants and Business Advisors (2025) *Taxation of Cryptocurrency in the UK: Upcoming Changes and What You Need to Know*. Available at: <https://r-m-t.co.uk/blog/taxation-of-cryptocurrency-upcoming-changes/> [Accessed: 3 June 2026]

John, D.L., Binnewies, S. and Stantic, B., 2024. Cryptocurrency price prediction algorithms: A survey and future directions. *Forecasting*, 6(3), pp.637-671.

Chen, J., 2023. Analysis of bitcoin price prediction using machine learning. *Journal of risk and financial management*, 16(1), p.51.

Kaur, R., Uppal, M., Gupta, D., Juneja, S., Arafat, S.Y., Rashid, J., Kim, J. and Alroobaea, R., 2025. Development of a cryptocurrency price prediction model: leveraging GRU and LSTM for Bitcoin, Litecoin and Ethereum. *PeerJ Computer Science*, 11, p.e2675.

Appendix

Critical Table (Appendix)

Author(s) & Year	Journal	What Was Done	Data Used	Methods	Key Findings	Relation to Project	Strengths	Limitations
Luo and Yu (2024)	Review of Accounting Studies (Springer)	Compare d GAAP and IFRS crypto accounting	40 global company financial statements	Comparative financial statement analysis	Persistent GAAP/IFRS inconsistencies in crypto cash flow classification	Evidence s UK IFRS accounting framework gaps	Large sample; top-tier journal	Mainly US companies; limited IFRS focus
Parrondo (2023)	Intelligent Systems in Accounting, Finance and Management (Wiley)	Proposed IFRS taxonomy for cryptoassets	EFRAG and MiCA regulation documents	Conceptual analytical framework	Payment tokens under IAS 38; security tokens under IFRS 9	Underpins this project's accounting classification analysis	Fills IFRS definitional vacuum	Conceptual only; untested empirically
Lazea, Bunget and Lungu (2024)	Risks (MDPI)	Bibliometric review of crypto accounting literature	157 manuscripts from WoS and Scopus (2007–2023)	Bibliometric analysis; VOSviewer; Biblioshiny	No universal IFRS classification; accounting ambiguity creates audit risk	Confirms accounting ambiguity central to research question	Dual accounting and audit focus; open access	No UK-specific accounting environment assessed
Baer et al. (2023)	Oxford Review of Economic Policy	Reviewed global cryptocurrency tax policy	HMRC, IRS and multi-jurisdictional tax data	Policy and economic review	Pseudonymity is structural enforcement obstacle; global CGT at stake	Supports HMRC enforcement limitation analysis	Prestigious journal; IMF-backed	No UK-specific CGT revenue modelling

Author(s) & Year	Journal	What Was Done	Data Used	Methods	Key Findings	Relation to Project	Strengths	Limitations
Bhullar et al. (2025)	Journal of Accounting Literature (Emerald)	Systematic bibliometric review of crypto taxation	Decade of published literature; VOSviewer; R-studio	Bibliometric analysis	Cross-jurisdictional tax discrepancies drive evasion and uncertainty	Validates this project's taxation research questions	Decade-spanning; Emerald peer-reviewed	Global scope; no primary HMRC data
Kicova et al. (2025)	Data Science in Finance and Economics (AIMS)	Systematic review of 38 crypto taxation studies	38 academic studies; 7-step methodology	Systematic literature review	Pseudonymous blockchain impedes income attribution; DeFi creates tax gaps	Supports argument HMRC self-reporting is insufficient	Rigorous methodology; ten thematic categories	Secondary literature only; no primary data
Hernandez Sanchez et al. (2024)	Economies (MDPI)	Empirical study of crypto tax ignorance and non-compliance	Survey data from Spanish crypto holders	Quantitative survey analysis	41.7% believe tax regulation discourages investment	Relevant to HMRC nudge letter behavioural approach	Primary empirical data; rare behavioural study	Spanish context; social desirability bias
Nembe et al. (2024)	Finance and Accounting Research Journal	Legal implications of blockchain for tax compliance	Legal frameworks across jurisdictions	Systematic legal analysis	Centralised regulation cannot govern decentralised blockchain	Supports FSMA 2023 gap and CARF analysis	Interdisciplinary; post-2023 recency	No cost modelling of non-compliance
Phung Duy et al. (2024)	Business Analyst Journal (Emerald)	ARIMA-GARCH models on Bitcoin daily prices	Bitcoin closing prices 2021–2023 (1,065 observations)	ARIMA-GARCH; AIC selection; Ljung-Box test	ARIMA-GARCH captures volatility clustering and non-stationarity	Computational benchmark for this project's EDA approach	Open access; rigorous diagnostics	Bitcoin only; excludes Ethereum and Litecoin

Code

```
# =====
# Effect of Digital Currency on Finance and Accounting:
# Challenges Associated with Taxation and Legislation in the UK
# =====

# =====
# SECTION 1: INSTALL AND IMPORT LIBRARIES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# Set plot style
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['axes.titlesize'] = 14
plt.rcParams['axes.labelsize'] = 12
plt.rcParams['xtick.labelsize'] = 10
plt.rcParams['ytick.labelsize'] = 10

# =====
# SECTION 2: LOAD DATA
# =====

btc = pd.read_csv('BITCOIN24.csv')
eth = pd.read_csv('ETHEREUM.csv')
ltc = pd.read_csv('LITECOIN24.csv')

print("Files loaded successfully.")
print(f"BTC rows: {len(btc)} | ETH rows: {len(eth)} | LTC rows: {len(ltc)}")

# =====
# SECTION 3: PRE-PROCESSING
# =====

# Step 1: Convert Date column to datetime format
btc['Date'] = pd.to_datetime(btc['Date'], dayfirst=True)
eth['Date'] = pd.to_datetime(eth['Date'], dayfirst=True)
ltc['Date'] = pd.to_datetime(ltc['Date'], dayfirst=True)

# Step 2: Convert ETH and LTC numeric columns from string to float
numeric_cols = ['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']
for col in numeric_cols:
    eth[col] = pd.to_numeric(eth[col], errors='coerce')
    ltc[col] = pd.to_numeric(ltc[col], errors='coerce')

# Step 3: Sort all files chronologically ascending
btc = btc.sort_values('Date').reset_index(drop=True)
eth = eth.sort_values('Date').reset_index(drop=True)
ltc = ltc.sort_values('Date').reset_index(drop=True)

# Step 4: Check and remove null values
```

```

print("\nNull values before cleaning:")
print(f"BTC: {btc.isnull().sum().sum()}")
print(f"ETH:
{eth[['Open', 'High', 'Low', 'Close', 'Volume']].isnull().sum().sum()}")
print(f"LTC:
{ltc[['Open', 'High', 'Low', 'Close', 'Volume']].isnull().sum().sum()}")

eth = eth.dropna(subset=['Close']).reset_index(drop=True)
ltc = ltc.dropna(subset=['Close']).reset_index(drop=True)

print("\nNull values after cleaning:")
print(f"ETH: {eth['Close'].isnull().sum()}")
print(f"LTC: {ltc['Close'].isnull().sum()}")

# Step 5: Check for duplicate dates
print(f"\nDuplicate dates - BTC:
{btc.duplicated(subset=['Date']).sum()}")
print(f"Duplicate dates - ETH: {eth.duplicated(subset=['Date']).sum()}")
print(f"Duplicate dates - LTC: {ltc.duplicated(subset=['Date']).sum()}")

# Step 6: Drop Adj Close (identical to Close)
eth = eth.drop(columns=['Adj Close'])
ltc = ltc.drop(columns=['Adj Close'])

print("\nPre-processing complete.")
print(f"Usable rows - BTC: {len(btc)} | ETH: {len(eth)} | LTC:
{len(ltc)}")

# =====
# SECTION 4: DATASET STRUCTURE SUMMARY
# =====

print("\n" + "="*60)
print("DATASET STRUCTURE SUMMARY")
print("="*60)

for name, df in [('Bitcoin', btc), ('Ethereum', eth), ('Litecoin', ltc)]:
    print(f"\n{name}:")
    print(f"  Date range : {df['Date'].min().date()} to
{df['Date'].max().date()}")
    print(f"  Total rows : {len(df)}")
    print(f"  Columns      : {list(df.columns)}")

# =====
# SECTION 5: DESCRIPTIVE STATISTICS (TABLE 1)
# =====

print("\n" + "="*60)
print("TABLE 1: DESCRIPTIVE STATISTICS - DAILY CLOSING PRICE (USD)")
print("="*60)

desc_stats = pd.DataFrame({
    'Bitcoin (BTC)': btc['Close'].describe(),
    'Ethereum (ETH)': eth['Close'].describe(),
    'Litecoin (LTC)': ltc['Close'].describe()
}).round(2)

skew_row = pd.DataFrame({
    'Bitcoin (BTC)': [round(btc['Close'].skew(), 4)],
    'Ethereum (ETH)': [round(eth['Close'].skew(), 4)],
    'Litecoin (LTC)': [round(ltc['Close'].skew(), 4)]
})

```

```

}, index=['skewness'])

kurt_row = pd.DataFrame({
    'Bitcoin (BTC)': [round(btc['Close'].kurtosis(), 4)],
    'Ethereum (ETH)': [round(eth['Close'].kurtosis(), 4)],
    'Litecoin (LTC)': [round(ltc['Close'].kurtosis(), 4)]
}, index=['kurtosis'])

desc_stats = pd.concat([desc_stats, skew_row, kurt_row])
print(desc_stats.to_string())

# =====
# SECTION 6: CLOSING PRICE TIME SERIES
# =====

fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Key regulatory event dates
regulatory_dates = {
    'Bitcoin Peak\n(Nov 2021)': '2021-11-10',
    'Market Crash\n(Nov 2022)': '2022-11-09',
    'FSMA 2023\n(Jun 2023)': '2023-06-29'
}

colors = ['#F7931A', '#627EEA', '#BFB BBB']
assets = [('Bitcoin (BTC)', btc, colors[0]),
          ('Ethereum (ETH)', eth, colors[1]),
          ('Litecoin (LTC)', ltc, colors[2])]

for ax, (name, df, color) in zip(axes, assets):
    ax.plot(df['Date'], df['Close'], color=color, linewidth=1.2,
            label=name)
    for label, date in regulatory_dates.items():
        dt = pd.Timestamp(date)
        if dt >= df['Date'].min() and dt <= df['Date'].max():
            ax.axvline(x=dt, color='red', linestyle='--', linewidth=0.8,
                        alpha=0.7)
            ax.text(dt, ax.get_ylim()[1] * 0.85, label,
                    fontsize=7, color='red', ha='center')
    ax.set_title(f'{name} - Daily Closing Price (USD)',
                 fontweight='bold')
    ax.set_ylabel('Price (USD)')
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    ax.xaxis.set_major_locator(mdates.YearLocator())
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper left', fontsize=9)

axes[2].set_xlabel('Date')
plt.suptitle('Daily Closing Price - Bitcoin, Ethereum and Litecoin (2018-2024)',
             fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

# =====
# SECTION 7: TRADING VOLUME TIME SERIES
# =====

fig, axes = plt.subplots(3, 1, figsize=(14, 12))

for ax, (name, df, color) in zip(axes, assets):

```

```

    ax.bar(df['Date'], df['Volume'], color=color, alpha=0.6, width=1.5,
label=name)
    for label, date in regulatory_dates.items():
        dt = pd.Timestamp(date)
        if dt >= df['Date'].min() and dt <= df['Date'].max():
            ax.axvline(x=dt, color='red', linestyle='--', linewidth=0.8,
alpha=0.7)
    ax.set_title(f'{name} - Daily Trading Volume', fontweight='bold')
    ax.set_ylabel('Volume (USD)')
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    ax.xaxis.set_major_locator(mdates.YearLocator())
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper left', fontsize=9)

axes[2].set_xlabel('Date')
plt.suptitle('Daily Trading Volume - Bitcoin, Ethereum and Litecoin
(2018-2024)',
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

# =====
# SECTION 8: BITCOIN MARKET CAPITALISATION
# =====

fig, ax = plt.subplots(figsize=(14, 5))
ax.fill_between(btc['Date'], btc['Market Cap'], color='#F7931A',
alpha=0.4)
ax.plot(btc['Date'], btc['Market Cap'], color='#F7931A', linewidth=1.2)

for label, date in regulatory_dates.items():
    dt = pd.Timestamp(date)
    ax.axvline(x=dt, color='red', linestyle='--', linewidth=0.8,
alpha=0.7)
    ax.text(dt, btc['Market Cap'].max() * 0.9, label,
            fontsize=8, color='red', ha='center')

ax.set_title('Bitcoin Market Capitalisation (USD) - 2018 to 2024',
            fontweight='bold')
ax.set_ylabel('Market Cap (USD)')
ax.set_xlabel('Date')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# =====
# SECTION 9: CORRELATION ANALYSIS (TABLE 2)
# =====

# Merge on overlapping dates
merged = btc[['Date', 'Close']].merge(
    eth[['Date', 'Close']], on='Date', suffixes=('_btc', '_eth')
).merge(ltc[['Date', 'Close']], on='Date')
merged.rename(columns={'Close': 'Close_ltc'}, inplace=True)

corr_matrix = merged[['Close_btc', 'Close_eth',
'Close_ltc']].corr().round(4)
corr_matrix.index = ['Bitcoin', 'Ethereum', 'Litecoin']
corr_matrix.columns = ['Bitcoin', 'Ethereum', 'Litecoin']

```

```

print("\n" + "="*60)
print("TABLE 2: PEARSON CORRELATION MATRIX - DAILY CLOSING PRICES")
print(f"(Overlapping period: {merged['Date'].min().date()} to {merged['Date'].max().date()}, n={len(merged)} days)")
print("="*60)
print(corr_matrix.to_string())

# CORRELATION HEATMAP
fig, ax = plt.subplots(figsize=(7, 5))
sns.heatmap(corr_matrix, annot=True, fmt='.4f', cmap='YlOrRd',
            linewidths=0.5, ax=ax, vmin=0, vmax=1,
            annot_kws={'size': 13, 'weight': 'bold'})
ax.set_title('Correlation Heatmap - Daily Closing Prices\n'
            f'(2018-01-01 to 2024-02-28, n={len(merged)} days)',
            fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# SECTION 11: VOLATILITY ANALYSIS - DAILY RETURNS (TABLE 3)
# =====

# Calculate daily log returns
btc['Log_Return'] = np.log(btc['Close'] / btc['Close'].shift(1))
eth['Log_Return'] = np.log(eth['Close'] / eth['Close'].shift(1))
ltc['Log_Return'] = np.log(ltc['Close'] / ltc['Close'].shift(1))

# Annualised volatility (standard deviation of log returns * sqrt(365))
btc_vol = btc['Log_Return'].std() * np.sqrt(365)
eth_vol = eth['Log_Return'].std() * np.sqrt(365)
ltc_vol = ltc['Log_Return'].std() * np.sqrt(365)

print("\n" + "="*60)
print("TABLE 3: VOLATILITY ANALYSIS - DAILY LOG RETURNS")
print("="*60)

vol_stats = pd.DataFrame({
    'Bitcoin (BTC)': [
        round(btc['Log_Return'].mean() * 100, 4),
        round(btc['Log_Return'].std() * 100, 4),
        round(btc['Log_Return'].min() * 100, 4),
        round(btc['Log_Return'].max() * 100, 4),
        round(btc_vol * 100, 2)
    ],
    'Ethereum (ETH)': [
        round(eth['Log_Return'].mean() * 100, 4),
        round(eth['Log_Return'].std() * 100, 4),
        round(eth['Log_Return'].min() * 100, 4),
        round(eth['Log_Return'].max() * 100, 4),
        round(eth_vol * 100, 2)
    ],
    'Litecoin (LTC)': [
        round(ltc['Log_Return'].mean() * 100, 4),
        round(ltc['Log_Return'].std() * 100, 4),
        round(ltc['Log_Return'].min() * 100, 4),
        round(ltc['Log_Return'].max() * 100, 4),
        round(ltc_vol * 100, 2)
    ]
}, index=['Mean Daily Return (%)', 'Std Dev (%)', 'Min Return (%)',
        'Max Return (%)', 'Annualised Volatility (%)'])

```

```

print(vol_stats.to_string())

# =====
# SECTION 12: DAILY LOG RETURNS
# =====

fig, axes = plt.subplots(3, 1, figsize=(14, 12))

return_colors = ['#F7931A', '#627EEA', '#BFBBBB']
return_assets = [('Bitcoin (BTC)', btc, return_colors[0]),
                  ('Ethereum (ETH)', eth, return_colors[1]),
                  ('Litecoin (LTC)', ltc, return_colors[2])]

for ax, (name, df, color) in zip(axes, return_assets):
    ax.plot(df['Date'], df['Log_Return'] * 100, color=color,
            linewidth=0.6, alpha=0.8, label=name)
    ax.axhline(y=0, color='black', linewidth=0.8, linestyle='-')
    for label, date in regulatory_dates.items():
        dt = pd.Timestamp(date)
        if dt >= df['Date'].min() and dt <= df['Date'].max():
            ax.axvline(x=dt, color='red', linestyle='--',
                      linewidth=0.8, alpha=0.7)
    ax.set_title(f'{name} - Daily Log Returns (%)', fontweight='bold')
    ax.set_ylabel('Log Return (%)')
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
    ax.xaxis.set_major_locator(mdates.YearLocator())
    ax.grid(True, alpha=0.3)
    ax.legend(loc='upper left', fontsize=9)

axes[2].set_xlabel('Date')
plt.suptitle('Daily Log Returns - Bitcoin, Ethereum and Litecoin (2018-2024)',
            fontsize=13, fontweight='bold', y=1.01)
plt.tight_layout()
plt.show()

# =====
# SECTION 13: CGT SIMULATION - OBJECTIVE 2
# Simulates annual CGT liability for a hypothetical UK investor
# holding a fixed quantity of each cryptocurrency
# Based on HMRC disposal rules: each year-end disposal is taxed
# at CGT rates with annual exempt amounts applied
# =====

print("\n" + "="*60)
print("TABLE 4: SIMULATED ANNUAL CGT LIABILITY - UK INVESTOR")
print("Assumptions: 1 BTC | 10 ETH | 100 LTC purchased Jan 2018")
print("Annual disposal at year-end closing price")
print("CGT rate: 20% (higher rate taxpayer)")
print("="*60)

# Quantities held
BTC_QTY = 1
ETH_QTY = 10
LTC_QTY = 100

# Cost basis (Jan 2018 opening prices)
BTC_COST = float(btc[btc['Date'] == '2018-01-01']['Open'].values[0])
ETH_COST = float(eth[eth['Date'] == '2018-01-01']['Open'].values[0])
LTC_COST = float(ltc[ltc['Date'] == '2018-01-01']['Open'].values[0])

```

```

# Annual exempt amounts per HMRC
exempt_amounts = {
    2018: 11700, 2019: 12000, 2020: 12300, 2021: 12300,
    2022: 12300, 2023: 6000, 2024: 3000
}

# CGT rate for higher rate taxpayer
CGT_RATE = 0.20

cgt_results = []

for year in range(2018, 2025):
    # Get year-end closing prices
    year_end = f'{year}-12-31'
    btc_year = btc[btc['Date'].dt.year == year]
    eth_year = eth[eth['Date'].dt.year == year]
    ltc_year = ltc[ltc['Date'].dt.year == year]

    if len(btc_year) == 0:
        continue

    btc_price = float(btc_year['Close'].iloc[-1])
    eth_price = float(eth_year['Close'].iloc[-1]) if len(eth_year) > 0
    else 0
    ltc_price = float(ltc_year['Close'].iloc[-1]) if len(ltc_year) > 0
    else 0

    # Calculate gains
    btc_gain = (btc_price - BTC_COST) * BTC_QTY
    eth_gain = (eth_price - ETH_COST) * ETH_QTY
    ltc_gain = (ltc_price - LTC_COST) * LTC_QTY
    total_gain = btc_gain + eth_gain + ltc_gain

    # Apply annual exempt amount
    exempt = exempt_amounts.get(year, 3000)
    taxable_gain = max(0, total_gain - exempt)
    cgt_liability = taxable_gain * CGT_RATE

    cgt_results.append({
        'Tax Year': f'{year}/{str(year+1)[-2:]}',
        'BTC Close (USD)': round(btc_price, 2),
        'ETH Close (USD)': round(eth_price, 2),
        'LTC Close (USD)': round(ltc_price, 2),
        'Total Gain (USD)': round(total_gain, 2),
        'CGT Exempt (USD)': exempt,
        'Taxable Gain (USD)': round(taxable_gain, 2),
        'CGT Liability (USD)': round(cgt_liability, 2)
    })

cgt_df = pd.DataFrame(cgt_results)
print(cgt_df.to_string(index=False))

# =====
# SECTION 14: CGT LIABILITY OVER TIME
# =====

fig, ax1 = plt.subplots(figsize=(12, 6))

x = range(len(cgt_df))
bars = ax1.bar(x, cgt_df['CGT Liability (USD)'],

```

```

        color=['#2ecc71' if v >= 0 else '#e74c3c'
               for v in cgt_df['CGT Liability (USD)']],
        alpha=0.8, label='CGT Liability (USD)')

ax1.set_xlabel('Tax Year')
ax1.set_ylabel('CGT Liability (USD)', color='black')
ax1.set_xticks(x)
ax1.set_xticklabels(cgt_df['Tax Year'], rotation=45)
ax1.grid(True, alpha=0.3, axis='y')

ax2 = ax1.twinx()
ax2.plot(x, cgt_df['CGT Exempt (USD)'], 'r--o',
         linewidth=2, markersize=6, label='Annual Exempt Amount (USD)')
ax2.set_ylabel('CGT Annual Exempt Amount (USD)', color='red')
ax2.tick_params(axis='y', labelcolor='red')

lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper left',
           fontsize=9)

plt.title('Simulated Annual CGT Liability vs Annual Exempt Amount\n'
          '(1 BTC + 10 ETH + 100 LTC, UK Higher Rate Taxpayer)',
          fontweight='bold')
plt.tight_layout()
plt.show()

# =====
# SECTION 15: REGULATORY EVENT IMPACT ANALYSIS - OBJECTIVE 3
# Examines price behaviour 30 days before and after key UK events
# =====

print("\n" + "="*60)
print("TABLE 5: PRICE IMPACT OF KEY UK REGULATORY EVENTS")
print("(Average closing price 30 days before vs 30 days after)")
print("="*60)

events = {
    'FSMA 2023 Royal Assent (29 Jun 2023)': '2023-06-29',
    'HMRC CGT Reduction (06 Apr 2023)': '2023-04-06',
    'Bitcoin Market Peak (10 Nov 2021)': '2021-11-10',
    'Crypto Market Crash (09 Nov 2022)': '2022-11-09'
}

event_results = []

for event_name, event_date in events.items():
    dt = pd.Timestamp(event_date)
    for asset_name, df in [('Bitcoin', btc), ('Ethereum', eth),
                           ('Litecoin', ltc)]:
        before = df[(df['Date'] >= dt - pd.Timedelta(days=30)) &
                     (df['Date'] < dt)][['Close']].mean()
        after = df[(df['Date'] > dt) &
                    (df['Date'] <= dt +
                     pd.Timedelta(days=30))][['Close']].mean()
        if pd.notna(before) and pd.notna(after):
            change_pct = ((after - before) / before) * 100
            event_results.append({
                'Event': event_name,
                'Asset': asset_name,
                'Avg Price Before (USD)': round(before, 2),

```

```

        'Avg Price After (USD)': round(after, 2),
        'Price Change (%)': round(change_pct, 2)
    })

event_df = pd.DataFrame(event_results)
print(event_df.to_string(index=False))

# =====
# SECTION 16: REGULATORY EVENT PRICE IMPACT
# =====

fig, ax = plt.subplots(figsize=(13, 6))

pivot = event_df.pivot(index='Event', columns='Asset', values='Price
Change (%)')
x = np.arange(len(pivot.index))
width = 0.25

bars1 = ax.bar(x - width, pivot['Bitcoin'], width,
               label='Bitcoin', color='#F7931A', alpha=0.85)
bars2 = ax.bar(x, pivot['Ethereum'], width,
               label='Ethereum', color='#627EEA', alpha=0.85)
bars3 = ax.bar(x + width, pivot['Litecoin'], width,
               label='Litecoin', color='#888888', alpha=0.85)

ax.axhline(y=0, color='black', linewidth=0.8)
ax.set_xlabel('Regulatory / Market Event')
ax.set_ylabel('Price Change (%)')
ax.set_title('Price Change 30 Days Before vs After Key UK Regulatory
Events',
             fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(pivot.index, rotation=15, ha='right', fontsize=9)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# =====
# SECTION 17: RISK AND OPPORTUNITY ANALYSIS - OBJECTIVE 4
# Rolling 30-day volatility and Sharpe Ratio
# =====

# Rolling 30-day volatility
btc['Rolling_Vol_30'] = btc['Log_Return'].rolling(window=30).std() *
np.sqrt(365) * 100
eth['Rolling_Vol_30'] = eth['Log_Return'].rolling(window=30).std() *
np.sqrt(365) * 100
ltc['Rolling_Vol_30'] = ltc['Log_Return'].rolling(window=30).std() *
np.sqrt(365) * 100

# Sharpe Ratio (risk-free rate = 2% annual, approximate UK base rate
average)
RISK_FREE_RATE = 0.02 / 365

btc_sharpe = (btc['Log_Return'].mean() - RISK_FREE_RATE) /
btc['Log_Return'].std()
eth_sharpe = (eth['Log_Return'].mean() - RISK_FREE_RATE) /
eth['Log_Return'].std()
ltc_sharpe = (ltc['Log_Return'].mean() - RISK_FREE_RATE) /
ltc['Log_Return'].std()

```

```

print("\n" + "="*60)
print("TABLE 6: RISK AND RETURN SUMMARY - OBJECTIVE 4")
print("="*60)

risk_df = pd.DataFrame({
    'Metric': [
        'Annualised Volatility (%)',
        'Annualised Return (%)',
        'Sharpe Ratio (daily)',
        'Max Drawdown (%)',
        'Value at Risk 95% (daily %)'
    ],
    'Bitcoin': [
        round(btc_vol * 100, 2),
        round(btc['Log_Return'].mean() * 365 * 100, 2),
        round(btc_sharpe, 4),
        round(((btc['Close'] / btc['Close'].cummax()) - 1).min() * 100,
2),
        round(btc['Log_Return'].quantile(0.05) * 100, 4)
    ],
    'Ethereum': [
        round(eth_vol * 100, 2),
        round(eth['Log_Return'].mean() * 365 * 100, 2),
        round(eth_sharpe, 4),
        round(((eth['Close'] / eth['Close'].cummax()) - 1).min() * 100,
2),
        round(eth['Log_Return'].quantile(0.05) * 100, 4)
    ],
    'Litecoin': [
        round(ltc_vol * 100, 2),
        round(ltc['Log_Return'].mean() * 365 * 100, 2),
        round(ltc_sharpe, 4),
        round(((ltc['Close'] / ltc['Close'].cummax()) - 1).min() * 100,
2),
        round(ltc['Log_Return'].quantile(0.05) * 100, 4)
    ]
})
print(risk_df.to_string(index=False))

# =====
# SECTION: MOVING AVERAGE FORECASTING - BITCOIN (OBJECTIVE 1)
# 30-day and 90-day Simple Moving Averages
# Informed by Phung Duy et al. (2024) time series methodology
# =====

btc['SMA_30'] = btc['Close'].rolling(window=30).mean()
btc['SMA_90'] = btc['Close'].rolling(window=90).mean()

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(btc['Date'], btc['Close'],
        color='#F7931A', linewidth=0.8, alpha=0.5, label='Actual Price')
ax.plot(btc['Date'], btc['SMA_30'],
        color='blue', linewidth=1.5, label='30-Day SMA')
ax.plot(btc['Date'], btc['SMA_90'],
        color='green', linewidth=1.5, label='90-Day SMA')
ax.set_title('Bitcoin - Simple Moving Average Analysis (30-Day and 90-
Day)',
            fontweight='bold')
ax.set_ylabel('Price (USD)')
ax.set_xlabel('Date')

```

```

ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# SMA Crossover Signal Table
btc['Signal'] = 0
btc.loc[btc['SMA_30'] > btc['SMA_90'], 'Signal'] = 1
btc.loc[btc['SMA_30'] < btc['SMA_90'], 'Signal'] = -1

crossovers = btc[btc['Signal'].diff() != 0][['Date', 'Close', 'SMA_30',
'SMA_90', 'Signal']].dropna()
crossovers['Signal'] = crossovers['Signal'].map({1: 'Bullish Cross', -1:
'Bearish Cross'})

print("\nTABLE 7: SMA CROSSOVER SIGNALS - BITCOIN")
print("(30-Day SMA crossing 90-Day SMA)")
print(crossovers.tail(10).to_string(index=False))

# =====
# SECTION 18: ROLLING 30-DAY VOLATILITY
# =====

fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(btc['Date'], btc['Rolling_Vol_30'],
        color='#F7931A', linewidth=1.2, label='Bitcoin', alpha=0.9)
ax.plot(eth['Date'], eth['Rolling_Vol_30'],
        color='#627EEA', linewidth=1.2, label='Ethereum', alpha=0.9)
ax.plot(ltc['Date'], ltc['Rolling_Vol_30'],
        color='#888888', linewidth=1.2, label='Litecoin', alpha=0.9)

for label, date in regulatory_dates.items():
    dt = pd.Timestamp(date)
    ax.axvline(x=dt, color='red', linestyle='--', linewidth=0.8,
alpha=0.7)
    ax.text(dt, ax.get_ylim()[1] * 0.9 if ax.get_ylim()[1] > 0 else 200,
            label, fontsize=7, color='red', ha='center')

ax.set_title('Rolling 30-Day Annualised Volatility (%) - 2018 to 2024',
            fontweight='bold')
ax.set_ylabel('Annualised Volatility (%)')
ax.set_xlabel('Date')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

##MONTHLY RETURN HEATMAP

def monthly_return_matrix(price_df):
    close_series = price_df.set_index('Date')['Close']
    month_end_prices = close_series.resample('ME').last()
    monthly_returns = month_end_prices.pct_change() * 100

    returns_frame = monthly_returns.to_frame('Return')
    returns_frame['Year'] = returns_frame.index.year

```

```

    returns_frame['Month'] = returns_frame.index.strftime('%b')

    pivot = returns_frame.pivot(index='Year', columns='Month',
values='Return')
    month_order = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
    pivot = pivot.reindex(columns=[m for m in month_order if m in
pivot.columns])
    return pivot

btc_monthly = monthly_return_matrix(btc)
eth_monthly = monthly_return_matrix(eth)
ltc_monthly = monthly_return_matrix(ltc)

print("\n" + "=" * 60)
print("TABLE: MONTHLY RETURN MATRIX - BITCOIN (%)")
print("=" * 60)
print(btc_monthly.round(2).to_string())

fig, axes = plt.subplots(3, 1, figsize=(13, 14))

asset_pivots = [('Bitcoin (BTC)', btc_monthly),
                ('Ethereum (ETH)', eth_monthly),
                ('Litecoin (LTC)', ltc_monthly)]

max_abs_return = max(pivot.abs().max().max() for _, pivot in
asset_pivots)

for ax, (name, pivot) in zip(axes, asset_pivots):
    sns.heatmap(pivot, annot=True, fmt='.1f', cmap='RdYlGn', center=0,
                vmin=-max_abs_return, vmax=max_abs_return,
linewidths=0.5, ax=ax,
                cbar_kws={'label': 'Monthly Return (%)'},
                annot_kws={'size': 8})
    ax.set_title(f'{name} - Monthly Return Heatmap (%)',
fontweight='bold')
    ax.set_ylabel('Year')
    ax.set_xlabel('')

axes[2].set_xlabel('Month')
plt.suptitle('Monthly Return Heatmap - Bitcoin, Ethereum and Litecoin
(2018-2024)',
            fontsize=13, fontweight='bold', y=1.005)
plt.tight_layout()
plt.show()

# =====
# SECTION 19: NORMALISED PRICE COMPARISON
# (Base 100 from Jan 2018 for direct comparison)
# =====

# Merge on common dates
common = btc[['Date', 'Close']].merge(
    eth[['Date', 'Close']], on='Date', suffixes=('_btc', '_eth')
).merge(ltc[['Date', 'Close']], on='Date')
common.rename(columns={'Close': 'Close_ltc'}, inplace=True)

# Normalise to base 100
common['BTC_norm'] = (common['Close_btc'] / common['Close_btc'].iloc[0])
* 100

```

```

common['ETH_norm'] = (common['Close_eth'] / common['Close_eth'].iloc[0])
* 100
common['LTC_norm'] = (common['Close_ltc'] / common['Close_ltc'].iloc[0])
* 100

fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(common['Date'], common['BTC_norm'],
        color='#F7931A', linewidth=1.5, label='Bitcoin')
ax.plot(common['Date'], common['ETH_norm'],
        color='#627EEA', linewidth=1.5, label='Ethereum')
ax.plot(common['Date'], common['LTC_norm'],
        color='#888888', linewidth=1.5, label='Litecoin')

ax.axhline(y=100, color='black', linewidth=0.8, linestyle='--',
alpha=0.5)

for label, date in regulatory_dates.items():
    dt = pd.Timestamp(date)
    ax.axvline(x=dt, color='red', linestyle='--', linewidth=0.8,
alpha=0.7)

ax.set_title('Normalised Price Performance (Base 100 = Jan 2018)\n'
            'Bitcoin vs Ethereum vs Litecoin', fontweight='bold')
ax.set_ylabel('Normalised Price (Base 100)')
ax.set_xlabel('Date')
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
ax.xaxis.set_major_locator(mdates.YearLocator())
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# SECTION 21: MACHINE LEARNING - FEATURE ENGINEERING
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (accuracy_score, precision_score,
recall_score,
                             f1_score, confusion_matrix)

RANDOM_STATE = 42

def compute_rsi(price_series, period=14):
    delta = price_series.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain = gain.rolling(period).mean()
    avg_loss = loss.rolling(period).mean()
    rs = avg_gain / avg_loss
    return 100 - (100 / (1 + rs))

def engineer_features(price_df, forward_window=10):
    data = price_df.copy()

    data['Log_Return'] = np.log(data['Close'] / data['Close'].shift(1))
    data['Return_1'] = data['Close'].pct_change(1)

```

```

data['Return_3'] = data['Close'].pct_change(3)
data['Return_7'] = data['Close'].pct_change(7)

data['Volatility_3'] = data['Log_Return'].rolling(3).std()
data['Volatility_7'] = data['Log_Return'].rolling(7).std()
data['Volatility_14'] = data['Log_Return'].rolling(14).std()

data['SMA_10'] = data['Close'].rolling(10).mean()
data['SMA_30'] = data['Close'].rolling(30).mean()
data['SMA_Ratio'] = data['SMA_10'] / data['SMA_30']
data['Price_SMA10_Ratio'] = data['Close'] / data['SMA_10']

data['RSI_14'] = compute_rsi(data['Close'], 14)

data['Volume_Change'] = data['Volume'].pct_change(1)
data['Volume_MA_Ratio'] = data['Volume'] /
data['Volume'].rolling(14).mean()

data['HL_Range'] = (data['High'] - data['Low']) / data['Close']
data['OC_Change'] = (data['Close'] - data['Open']) / data['Open']
data['Momentum_10'] = (data['Close'] - data['Close'].shift(10)) /
data['Close'].shift(10)

data['AbsReturn_1'] = data['Return_1'].abs()
data['AbsReturn_3'] = data['Return_1'].rolling(3).apply(lambda x:
np.abs(x).mean())

rolling_std_20 = data['Close'].rolling(20).std()
rolling_mean_20 = data['Close'].rolling(20).mean()
data['BB_Width'] = (4 * rolling_std_20) / rolling_mean_20
data['ATR_14'] = (data['High'] - data['Low']).rolling(14).mean() /
data['Close']

forward_volatility = (data['Log_Return'].shift(-1)
                      .rolling(forward_window).std()
                      .shift(-(forward_window - 1)))
data['Target'] = (forward_volatility >
data['Volatility_7']).astype(int)

data = data.replace([np.inf, -np.inf],
np.nan).dropna().reset_index(drop=True)
return data

FEATURE_COLUMNS = [
    'Return_1', 'Return_3', 'Return_7',
    'Volatility_3', 'Volatility_7', 'Volatility_14',
    'SMA_Ratio', 'Price_SMA10_Ratio', 'RSI_14',
    'Volume_Change', 'Volume_MA_Ratio',
    'HL_Range', 'OC_Change', 'Momentum_10',
    'AbsReturn_1', 'AbsReturn_3', 'BB_Width', 'ATR_14'
]

btc_ml = engineer_features(btc)
eth_ml = engineer_features(eth)
ltc_ml = engineer_features(ltc)

print("\n" + "=" * 60)
print("TABLE 8: FEATURE-ENGINEERED DATASET SIZE AND CLASS BALANCE")
print("=" * 60)

```

```

for asset_name, asset_df in [('Bitcoin', btc_ml), ('Ethereum', eth_ml),
                              ('Litecoin', ltc_ml)]:
    print(f"{asset_name:10s} rows={len(asset_df):5d} "
          f"positive class share={asset_df['Target'].mean():.3f}")

# =====
# SECTION 22: TRAIN-TEST SPLIT (CHRONOLOGICAL, 80/20)
# =====

def chronological_split(feature_df, test_size=0.2):
    split_point = int(len(feature_df) * (1 - test_size))
    X = feature_df[FEATURE_COLUMNS]
    y = feature_df['Target']
    X_train, X_test = X.iloc[:split_point], X.iloc[split_point:]
    y_train, y_test = y.iloc[:split_point], y.iloc[split_point:]
    return X_train, X_test, y_train, y_test

splits = {
    'Bitcoin': chronological_split(btc_ml),
    'Ethereum': chronological_split(eth_ml),
    'Litecoin': chronological_split(ltc_ml)
}

# =====
# SECTION 23: HYPERPARAMETER TUNING AND MODEL TRAINING
# GridSearchCV with TimeSeriesSplit cross-validation is used
# for all three models, so hyperparameter selection never
# uses information from later time points to evaluate earlier
# ones.
# =====

tscv = TimeSeriesSplit(n_splits=5)

logistic_param_grid = {
    'C': [0.01, 0.1, 0.5, 1, 5],
    'penalty': ['l2'],
    'solver': ['lbfgs']
}

random_forest_param_grid = {
    'n_estimators': [200, 300],
    'max_depth': [4, 6, 8],
    'min_samples_leaf': [3, 5, 10]
}

xgboost_param_grid = {
    'n_estimators': [200, 300],
    'max_depth': [2, 3, 4],
    'learning_rate': [0.01, 0.03, 0.05]
}

model_results = []
fitted_models = {}
confusion_matrices = {}

for asset_name, (X_train, X_test, y_train, y_test) in splits.items():
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

```

```

logistic_search = GridSearchCV(
    LogisticRegression(max_iter=2000, random_state=RANDOM_STATE),
    logistic_param_grid, cv=tscv, scoring='accuracy', n_jobs=-1
)
logistic_search.fit(X_train_scaled, y_train)
logistic_model = logistic_search.best_estimator_
logistic_pred = logistic_model.predict(X_test_scaled)

forest_search = GridSearchCV(
    RandomForestClassifier(random_state=RANDOM_STATE),
    random_forest_param_grid, cv=tscv, scoring='accuracy', n_jobs=-1
)
forest_search.fit(X_train, y_train)
forest_model = forest_search.best_estimator_
forest_pred = forest_model.predict(X_test)

xgboost_search = GridSearchCV(
    XGBClassifier(eval_metric='logloss', random_state=RANDOM_STATE),
    xgboost_param_grid, cv=tscv, scoring='accuracy', n_jobs=-1
)
xgboost_search.fit(X_train, y_train)
xgboost_model = xgboost_search.best_estimator_
xgboost_pred = xgboost_model.predict(X_test)

fitted_models[asset_name] = {
    'Logistic Regression': (logistic_model, logistic_pred,
logistic_search.best_params_, scaler),
    'Random Forest': (forest_model, forest_pred,
forest_search.best_params_, None),
    'XGBoost': (xgboost_model, xgboost_pred,
xgboost_search.best_params_, None)
}

for model_name, (fitted_model, predictions, best_params, _) in
fitted_models[asset_name].items():
    model_results.append({
        'Asset': asset_name,
        'Model': model_name,
        'Accuracy': round(accuracy_score(y_test, predictions), 4),
        'Precision': round(precision_score(y_test, predictions), 4),
        'Recall': round(recall_score(y_test, predictions), 4),
        'F1 Score': round(f1_score(y_test, predictions), 4),
        'Best Parameters': best_params
    })
    confusion_matrices[(asset_name, model_name)] =
confusion_matrix(y_test, predictions)

results_df = pd.DataFrame(model_results)

print("\n" + "=" * 60)
print("TABLE 9: MODEL COMPARISON - ACCURACY, PRECISION, RECALL, F1")
print("=" * 60)
print(results_df[['Asset', 'Model', 'Accuracy', 'Precision', 'Recall',
'F1 Score']].to_string(index=False))

print("\n" + "=" * 60)
print("TABLE 10: BEST HYPERPARAMETERS PER MODEL")
print("=" * 60)
for row in model_results:

```

```

    print(f"{row['Asset']:10s} | {row['Model']:20s} | {row['Best  
Parameters']}]")

# =====
# SECTION 24: MODEL ACCURACY COMPARISON
# =====

fig, ax = plt.subplots(figsize=(12, 6))

assets_order = ['Bitcoin', 'Ethereum', 'Litecoin']
models_order = ['Logistic Regression', 'Random Forest', 'XGBoost']
model_colors = {'Logistic Regression': '#4C72B0', 'Random Forest':  

'#55A868', 'XGBoost': '#C44E52'}

bar_width = 0.25
x_positions = np.arange(len(assets_order))

for i, model_name in enumerate(models_order):
    accuracies = [
        results_df[(results_df['Asset'] == a) & (results_df['Model'] ==  
model_name)][['Accuracy']].values[0]
        for a in assets_order
    ]
    ax.bar(x_positions + (i - 1) * bar_width, accuracies,  
width=bar_width,  
label=model_name, color=model_colors[model_name], alpha=0.9)

ax.axhline(y=0.6, color='black', linestyle='--', linewidth=1, alpha=0.6)
ax.set_xticks(x_positions)
ax.set_xticklabels(assets_order)
ax.set_ylabel('Test Set Accuracy')
ax.set_ylim(0, 1)
ax.set_title('Model Accuracy Comparison - Volatility Regime  
Classification', fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

# =====
# SECTION 25: CONFUSION MATRICES
# =====

fig, axes = plt.subplots(3, 3, figsize=(13, 12))

for row_idx, asset_name in enumerate(assets_order):
    for col_idx, model_name in enumerate(models_order):
        ax = axes[row_idx, col_idx]
        cm = confusion_matrices[(asset_name, model_name)]
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,  
ax=ax,  
xticklabels=['Low Vol', 'High Vol'],  
yticklabels=['Low Vol', 'High Vol'])
        ax.set_title(f'{asset_name} - {model_name}', fontsize=10,  
fontweight='bold')
        ax.set_ylabel('Actual')
        ax.set_xlabel('Predicted')

plt.suptitle('Confusion Matrices - All Assets and Models', fontsize=14,  
fontweight='bold', y=1.01)
plt.tight_layout()

```

```

plt.show()

# =====
# SECTION 26: FEATURE IMPORTANCE (RANDOM FOREST AND XGBOOST)
# =====

fig, axes = plt.subplots(3, 2, figsize=(13, 14))

for row_idx, asset_name in enumerate(assets_order):
    forest_model = fitted_models[asset_name]['Random Forest'][0]
    xgboost_model = fitted_models[asset_name]['XGBoost'][0]

    forest_importance = pd.Series(forest_model.feature_importances_,
index=FEATURE_COLUMNS)
    forest_importance =
forest_importance.sort_values(ascending=True).tail(10)

    xgboost_importance = pd.Series(xgboost_model.feature_importances_,
index=FEATURE_COLUMNS)
    xgboost_importance =
xgboost_importance.sort_values(ascending=True).tail(10)

    axes[row_idx, 0].barh(forest_importance.index,
forest_importance.values, color='#55A868')
    axes[row_idx, 0].set_title(f'{asset_name} - Random Forest',
fontsize=10, fontweight='bold')
    axes[row_idx, 0].set_xlabel('Importance')

    axes[row_idx, 1].barh(xgboost_importance.index,
xgboost_importance.values, color='#C44E52')
    axes[row_idx, 1].set_title(f'{asset_name} - XGBoost', fontsize=10,
fontweight='bold')
    axes[row_idx, 1].set_xlabel('Importance')

plt.suptitle('Feature Importance by Asset and Model', fontsize=14,
fontweight='bold', y=1.005)
plt.tight_layout()
plt.show()

# =====
# SECTION 27: TABLE 11 - BEST MODEL PER ASSET
# =====

best_model_summary = (results_df.sort_values('Accuracy', ascending=False)
                        .groupby('Asset').first()
                        .reset_index()[['Asset', 'Model', 'Accuracy', 'F1
Score']])

print("\n" + "=" * 60)
print("TABLE 11: HIGHEST-ACCURACY MODEL PER ASSET")
print("=" * 60)
print(best_model_summary.to_string(index=False))

# =====
# SECTION 29: LSTM PRICE FORECASTING - FEATURE PREPARATION
# =====

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
import tensorflow as tf

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

tf.random.set_seed(42)
np.random.seed(42)

LOOKBACK_WINDOW = 60
TEST_SIZE = 0.2

def build_sequences(scaled_series, window):
    X, y = [], []
    for i in range(window, len(scaled_series)):
        X.append(scaled_series[i - window:i, 0])
        y.append(scaled_series[i, 0])
    return np.array(X), np.array(y)

def prepare_lstm_data(price_df, window=LOOKBACK_WINDOW,
test_size=TEST_SIZE):
    close_prices = price_df['Close'].values.reshape(-1, 1)
    split_point = int(len(close_prices) * (1 - test_size))

    train_raw = close_prices[:split_point]
    test_raw = close_prices[split_point:]

    scaler = MinMaxScaler(feature_range=(0, 1))
    train_scaled = scaler.fit_transform(train_raw)

    test_input_raw = close_prices[split_point - window:]
    test_scaled = scaler.transform(test_input_raw)

    X_train, y_train = build_sequences(train_scaled, window)
    X_test, y_test = build_sequences(test_scaled, window)

    X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
    X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

    return X_train, y_train, X_test, y_test, scaler, train_raw, test_raw

# =====
# SECTION 30: LSTM MODEL ARCHITECTURE AND TRAINING
# =====

def build_lstm_model(window=LOOKBACK_WINDOW):
    model = Sequential([
        LSTM(64, return_sequences=True, input_shape=(window, 1)),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(16, activation='relu'),
        Dense(1)
    ])
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model

lstm_forecasts = {}
lstm_metrics = []

```

```

for asset_name, asset_df in [('Bitcoin', btc), ('Ethereum', eth),
                              ('Litecoin', ltc)]:

    X_train, y_train, X_test, y_test, scaler, train_raw, test_raw =
prepare_lstm_data(asset_df)

    lstm_model = build_lstm_model()
    early_stop = EarlyStopping(monitor='val_loss', patience=5,
restore_best_weights=True)
    lstm_model.fit(X_train, y_train, epochs=25, batch_size=64,
                    validation_split=0.1, verbose=0,
callbacks=[early_stop])

    predicted_scaled = lstm_model.predict(X_test, verbose=0)
    predicted_prices =
scaler.inverse_transform(predicted_scaled).flatten()
    actual_prices = scaler.inverse_transform(y_test.reshape(-1,
1)).flatten()

    naive_predicted = test_raw[LOOKBACK_WINDOW - 1:-1].flatten()
    naive_actual = test_raw[LOOKBACK_WINDOW:].flatten()

    test_start_index = len(asset_df) - len(test_raw)
    sma_series = asset_df['Close'].rolling(30).mean().values
    sma_predicted = sma_series[test_start_index - 1: len(asset_df) - 1]
    sma_actual = asset_df['Close'].values[test_start_index:]
    valid_sma = ~np.isnan(sma_predicted)

    lstm_forecasts[asset_name] = {
        'dates': asset_df['Date'].values[-len(actual_prices):],
        'actual': actual_prices,
        'lstm_predicted': predicted_prices
    }

    lstm_metrics.append({
        'Asset': asset_name,
        'Model': 'LSTM',
        'RMSE': round(np.sqrt(mean_squared_error(actual_prices,
predicted_prices)), 2),
        'MAE': round(mean_absolute_error(actual_prices,
predicted_prices), 2),
        'R2': round(r2_score(actual_prices, predicted_prices), 4),
        'MAPE (%)': round(np.mean(np.abs((actual_prices -
predicted_prices) / actual_prices)) * 100, 2)
    })

    lstm_metrics.append({
        'Asset': asset_name,
        'Model': 'Naive Persistence',
        'RMSE': round(np.sqrt(mean_squared_error(naive_actual,
naive_predicted)), 2),
        'MAE': round(mean_absolute_error(naive_actual, naive_predicted),
2),
        'R2': round(r2_score(naive_actual, naive_predicted), 4),
        'MAPE (%)': round(np.mean(np.abs((naive_actual - naive_predicted)
/ naive_actual)) * 100, 2)
    })

    lstm_metrics.append({
        'Asset': asset_name,
        'Model': '30-Day SMA',

```

```

        'RMSE': round(np.sqrt(mean_squared_error(sma_actual[valid_sma],
sma_predicted[valid_sma])), 2),
        'MAE': round(mean_absolute_error(sma_actual[valid_sma],
sma_predicted[valid_sma]), 2),
        'R2': round(r2_score(sma_actual[valid_sma],
sma_predicted[valid_sma]), 4),
        'MAPE (%)': round(np.mean(np.abs((sma_actual[valid_sma] -
sma_predicted[valid_sma])
                                / sma_actual[valid_sma])) *
100, 2)
    })

lstm_results_df = pd.DataFrame(lstm_metrics)

print("\n" + "=" * 60)
print("TABLE 12: LSTM FORECAST PERFORMANCE VS BASELINES")
print("=" * 60)
print(lstm_results_df.to_string(index=False))

# =====
# SECTION 31: ACTUAL VS LSTM-PREDICTED PRICE
# =====

fig, axes = plt.subplots(3, 1, figsize=(14, 13))

asset_colors = {'Bitcoin': '#F7931A', 'Ethereum': '#627EEA', 'Litecoin':
'#888888'}

for ax, asset_name in zip(axes, ['Bitcoin', 'Ethereum', 'Litecoin']):
    forecast_data = lstm_forecasts[asset_name]
    ax.plot(forecast_data['dates'], forecast_data['actual'],
            color='black', linewidth=1.2, label='Actual Close')
    ax.plot(forecast_data['dates'], forecast_data['lstm_predicted'],
            color=asset_colors[asset_name], linewidth=1.2, alpha=0.9,
label='LSTM Forecast')
    ax.set_title(f'{asset_name} - Actual vs LSTM-Predicted Closing Price
(Test Period)',
                fontweight='bold')
    ax.set_ylabel('Price (USD)')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.3)

axes[2].set_xlabel('Date')
plt.tight_layout()
plt.show()

# =====
# SECTION 32: RMSE COMPARISON ACROSS MODELS
# =====

fig, ax = plt.subplots(figsize=(11, 6))

assets_order = ['Bitcoin', 'Ethereum', 'Litecoin']
models_order = ['LSTM', 'Naive Persistence', '30-Day SMA']
comparison_colors = {'LSTM': '#4C72B0', 'Naive Persistence': '#55A868',
'30-Day SMA': '#C44E52'}

bar_width = 0.25
x_positions = np.arange(len(assets_order))

for i, model_name in enumerate(models_order):

```

```

    rmse_values = [
        lstm_results_df[(lstm_results_df['Asset'] == a) &
        (lstm_results_df['Model'] == model_name)][['RMSE']].values[0]
        for a in assets_order
    ]
    normalised_rmse = [
        rmse_values[j] / lstm_results_df[lstm_results_df['Asset'] ==
assets_order[j]][['RMSE']].max()
        for j in range(len(assets_order))
    ]
    ax.bar(x_positions + (i - 1) * bar_width, normalised_rmse,
width=bar_width,
        label=model_name, color=comparison_colors[model_name],
alpha=0.9)

ax.set_xticks(x_positions)
ax.set_xticklabels(assets_order)
ax.set_ylabel('RMSE (normalised to worst model per asset)')
ax.set_title('Forecast Error Comparison - LSTM vs Naive vs SMA Baseline',
fontweight='bold')
ax.legend(fontsize=9)
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()

```